



Universidade Federal do Ceará – UFC
Centro de Ciências – CC
Departamento de Computação - DC
Sistemas de Gerenciamento de Bancos de Dados



Exercício: Gerenciamento de Buffer

Objetivos: Introduzir os principais algoritmos de gerenciamento de buffer.

Data da Entrega: 31/05/2026

OBS 1: Exercício Individual.

OBS 2: A entrega da lista deverá ser executada utilizando-se o SIGAA.

NOME: _____ MATRÍCULA: _____

Questão 1

Implementar um dos seguintes algoritmos de gerenciamento de buffer no framework Seal-DB (<https://github.com/ggustavo/seal-db>):

- 1-) LFU-DA
- 2-) FBR
- 3-) LRU-K ($k = 2$)
- 4-) LRU-MIS
- 5-) GCLOCK
- 6-) MQ
- 7-) LIRS
- 8-) CFLRU
- 9-) LRU-WSR
- 10-) CCF-LRU
- 11-) CFDC
- 12-) CASA
- 13-) AD-LRU
- 14-) LLRU

A avaliação deste trabalho se dará em duas etapas:

1ª. Vídeo de Apresentação: Cada estudante irá disponibilizar um vídeo (no Youtube) apresentando o todo o código gerado, bem como as ferramentas utilizadas. O estudante pode utilizar slides e notebooks na produção do vídeo.

2ª. Avaliação do Código: O professor da disciplina irá avaliar a qualidade dos códigos (notebooks) gerados pelo estudante, bem como a utilização das ferramentas utilizadas e as análises realizadas.

A avaliação do trabalho irá envolver os seguintes quesitos:

- Qualidade e organização do código;
- Clareza da descrição das atividades realizadas e dos resultados obtidos;
- Domínio do tema;

PS. Não serão aceitos trabalhos que não forem apresentados (por meio de vídeo disponibilizado no Youtube).

PS. Cada estudante será responsável pela disponibilização do ambiente (software e hardware) necessário para a gravação da apresentação do seu trabalho.

Os arquivos de código fonte e a URL do vídeo deverão ser disponibilizados, em formato .ZIP, no SIGAA. Caso o tamanho do arquivo .ZIP ultrapasse o limite máximo permitido pelo SIGAA, o estudante pode disponibilizar um link para um repositório no Google Drive.

“Se não posso estimular sonhos impossíveis, não devo negar o direito de sonhar com quem sonha.”.

Paulo Freire

2 FUNDAMENTAÇÃO TEÓRICA

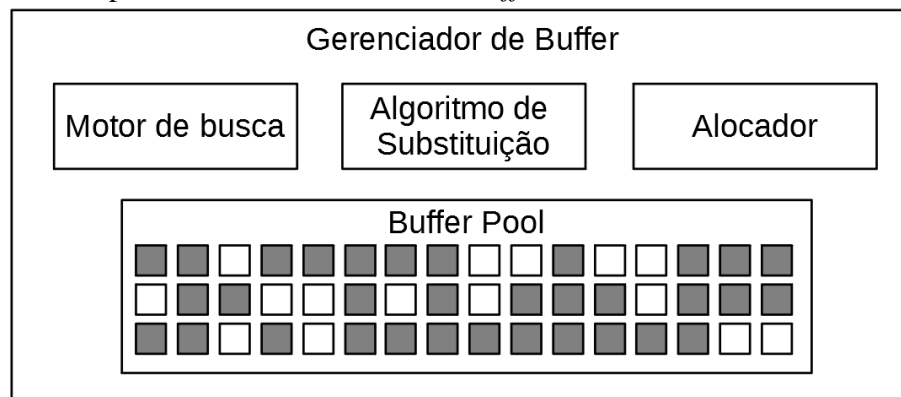
Este capítulo apresenta a fundamentação teórica de gerenciamento de *buffer* em SGBD. A Seção 2.1 apresenta uma visão geral da arquitetura dos gerenciadores de *buffer* de banco de dados. A Seção 2.2 discorre sobre alguns dos principais fundamentos das políticas de substituição de *buffer*. Problemas inerentes do gerenciamento de *buffer* em SGBDs são descritos na Seção 2.3. O impacto do uso de mídias assimétricas para o gerenciador de *buffer* é estudado na Seção 2.4. A Seção 2.5 brevemente descreve um algoritmo de substituição de referência, que por ser ótimo, serve como parâmetros de comparação em avaliação empírica de políticas de substituição de *buffer*. Por fim, a Seção 2.6 conclui este capítulo.

2.1 Arquitetura dos gerenciadores de *buffer*

Os SGBDs certamente são um dos sistemas de software mais complexos, pois devem implementar diversos requisitos funcionais e não funcionais para atender usuários e aplicações. Como discutido na Seção 1.1, SGBDs são implementados com diversos componentes que trabalham juntos para prover tais requisitos. Dentre estes componentes, encontra-se o gerenciador de *buffer* que pode apresentar um elevado custo computacional em relação aos demais componentes (ver Seção 1.2).

O gerenciador de *buffer* é o componente de software responsável por solicitar blocos ao gerenciador de arquivos para manter uma cópia desses blocos na memória principal (chamada página), melhorando significativamente o desempenho do SGBD. No contexto tecnológico atual, ainda há uma grande diferença do tempo de acesso aos dados entre a memória principal e as mídias de armazenamento secundárias, ficando evidente que, para um SGBD tradicional atingir um desempenho ideal, deve ser maximizado o número de requisições de página atendidas pelo gerenciador de *buffer* sem a necessidade de acessos aos dispositivos de armazenamento secundários. Como nem sempre é possível manter todas as páginas do banco de dados disponíveis na memória principal, também é função do gerenciador de *buffer* determinar qual o melhor subconjunto de páginas que deve residir na memória principal para atender as requisições de maneira eficiente.

A Figura 4 apresenta alguns subcomponentes do gerenciador de *buffer*. O subcomponente motor de busca é responsável por encontrar as páginas solicitadas pelos componentes de alto nível. Uma estratégia simples seria realizar uma busca sequencial para todas as páginas que

Figura 4 – Subcomponentes do Gerenciador de *buffer*

Fonte: O Autor.

estão atualmente alocadas no *buffer*, mas o custo e a frequência dessa operação podem ter um impacto negativo significativo no desempenho do SGBD. Portanto, realizar pesquisas usando tabelas de hash ou outros algoritmos de indexação são boas alternativas para encontrar as páginas solicitadas (EFFELSBURG; HÄRDER, 1984).

Quando o SGBD inicializa o gerenciador de *buffer* solicita um espaço de memória principal para alocar páginas através do sistema operacional. Este espaço é chamado de *Buffer Pool*. Em termos práticos de implementação, é o espaço de memória alocado para páginas de dados, geralmente é um valor fixo. Esse espaço também pode ter limites inferiores ou superiores, portanto, a alocação de memória das páginas deve variar apenas dentro desses limites. Na Figura 4, o Alocaador representa uma série de funções responsáveis por inicializar e manipular as estruturas de dados utilizadas no gerenciador de *buffer* e alocar os dados no *Buffer Pool*.

O *Buffer Pool* é implementado usando uma matriz de *frames* (região física da memória para alocação de blocos) onde cada *frames* é preferencialmente do tamanho de um bloco de mídia de armazenamento. Associado à matriz de *frames*, temos o subcomponente motor de busca que mapeia todos os blocos e indica a localização de seus respectivos frames no *Buffer Pool*. Portanto, as páginas são os frames (com seus respectivos dados de blocos) e com descritores que fornecem metadados sobre os dados alocados nos *frames*, como, por exemplo, o *dirty bit*, que indica que a página foi modificada (HELLERSTEIN *et al.*, 2007).

Também é importante notar que existem dois tipos de páginas, leitura (*clean*) ou escrita (*dirty*). Quando os outros componentes do SGBD requisitam páginas do gerenciador de *buffer*, eles indicam que tipo de operação eles executarão. Um bloco só é gravado de volta na mídia de armazenamento se uma operação de escrita o modificou no *Buffer Pool* (i.e., em memória principal). O gerenciador de *buffer* fornece um serviço de propagação de dados

local para componentes de nível superior, como o processador de consultas. Uma página pode ser reutilizada indefinidamente por vários aplicativos sem sempre buscá-la na mídia de armazenamento para cada acesso solicitado. Isto é chamado de *reentrância de dados*. Por esse motivo, o gerenciador de *buffer* melhora o desempenho do SGBD.

Outra vantagem do gerenciador de *buffer* é que os componentes de nível superior de um SGBD não precisam necessariamente saber se a página está na memória ou armazenada em mídia física. Isso simplifica o trabalho desses componentes, pois eles podem assumir que todo o banco de dados está na memória e precisam apenas solicitar páginas com suas operações. Por exemplo, se o processador de consultas precisa fazer uma varredura completa em uma tabela (operações de leitura de dados) ele pode consultar os metadados do catálogo para saber qual ID de página (identificador de página) será solicitado ao gerenciador de *buffer*. Dentre as informações usadas por estes componentes de nível superior temos (GARCIA-MOLINA *et al.*, 2001):

- Dados: o conteúdo armazenado no banco de dados por aplicações e usuários.
- Metadados: as informações que descrevem a estrutura do banco de dados e restrições sobre ele.
- Estatísticas: as informações obtidas pelo SGBD sobre propriedades específicas dos dados.
- Índices: as estruturas de dados que provem um acesso mais eficiente as informações do banco de dados.

O *Buffer Pool* do gerenciador de *buffer* tem um número limitado de páginas. Como resultado, pode haver solicitações de páginas que não estejam na memória principal. Suponha que uma página específica já esteja na memória (*hit*). É um cenário perfeito, pois não há custo de acesso à mídia de armazenamento físico para encontrá-la. No entanto, suponha que a página não esteja na memória (*miss*). Nesse caso, o gerenciador de *buffer* solicita os dados do componente gerenciador de arquivos e coloca a página no *Buffer Pool*, tendo um alto custo de acesso aos dados na mídia de armazenamento secundária.

O gerenciador de *buffer* usa o gerenciador de arquivos para mover blocos da mídia de armazenamento secundário para o *Buffer Pool*. No entanto, suponha que o *Buffer Pool* esteja cheio, sem espaço suficiente para alocar a página solicitada. Nesse caso, o gerenciador de *buffer* utiliza um recurso chamado política de substituição de página, que tem como objetivo determinar quais páginas permanecem ou são desaloçadas (despejadas) da memória. O subcomponente algoritmo de substituição escolhe uma vítima com base em estratégias que determinam a página

com o valor mais baixo, no nível do *buffer*, atualmente em memória. As estratégias de tomada de decisão de vítimas são baseadas no princípio da localidade, que se divide em localidade temporal e espacial.

- Localidade espacial: Na maioria das vezes, sempre que possível, é vantajoso armazenar sequencialmente blocos em um mesmo arquivo na mídia de armazenamento, pois isso facilita os mecanismos de hardware de busca de blocos. Por exemplo, ao solicitar a leitura completa deste arquivo específico, implica uma leitura sequencial de cada bloco. A localidade espacial demonstra maior probabilidade de acessar dados próximos aos dados solicitados. Assim, após um bloco ser acessado, há uma alta probabilidade de que o próximo bloco solicitado seja seu vizinho armazenado fisicamente.
- Localidade temporal: Diz respeito ao uso frequente de uma página/bloco. Quando um bloco específico é acessado várias vezes, há uma probabilidade maior de ser acessado novamente. É típico que um processador de consultas tenha algoritmos que possam acessar as mesmas páginas repetidamente em um curto período, portanto, é uma vantagem manter essas páginas frequentes na memória principal.

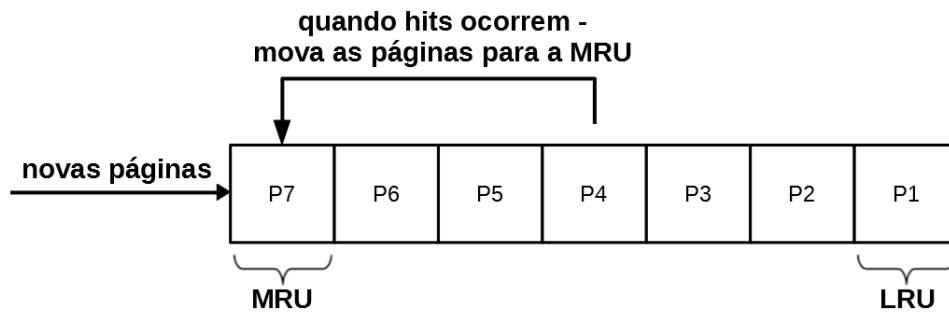
Alguns algoritmos de substituição de página consideram apenas a localidade temporal, tendo a frequência de acesso como seu principal recurso na sua tomada de decisões, isso pode agravar problemas quando um comportamento de acesso segue o princípio da localidade espacial (ver Seção 2.3.1).

Os algoritmos de substituição podem tirar proveito da técnica de pré-paginação que consistem em ler conjunto de blocos armazenados fisicamente próximos na mídia secundária em vez de ler apenas um bloco por vez, aproveitando ao máximo a largura de banda de acesso do dispositivo (EFFELSBURG; HARDER, 1984). Pré-paginação favorece o padrão de acesso que tende a seguir o princípio da localidade espacial, já que blocos pré-carregados já estão previamente no *Buffer Pool* esperando para serem requisitados. Porém, existe uma desvantagem quando temos um padrão de acesso que tende a seguir o princípio da localidade temporal, a probabilidade que vários blocos pré-carregados não sejam usados pelas transações é muito alta ocupando espaço no *Buffer Pool*. O uso ou não da técnica de pré-paginação afeta diretamente a performance do algoritmo de substituição de página. Isso gera uma decisão arquitetural durante a implementação de um gerenciador de *buffer*.

2.2 Visão geral das políticas de substituição de *buffer*

Para construir um algoritmo de substituição de páginas eficiente é essencial conhecer as operações do banco de dados, tanto as operações realizadas no momento da solicitação quanto as que podem vir. Uma estratégia ideal para todos os cenários é desconhecida. Vários fatores internos e subjacentes ao gerenciador de *buffer* podem influenciar a ordem e o entrelaçamento das operações (SILBERSCHATZ *et al.*, 2005). Em qualquer caso, uma política de substituição adequada deve maximizar o reuso de dados, para que, na grande maioria das vezes que uma página é solicitada ela já resida no *Buffer Pool* (na memória). A seguir, apresentamos algumas das políticas básicas utilizadas na construção de políticas sofisticadas.

Figura 5 – Fluxo de execução da LRU e MRU



Fonte: O Autor.

Least-Recently Used (LRU) e *Most-Recently Used* (MRU) são políticas de substituição de página que podem ser implementadas usando uma lista, seguindo a ordem das referências. A figura 5 demonstra o fluxo de execução. Quando uma página é acessada, ela é movida para o topo da lista (MRU), indicando que foi a última página referenciada. A mesma coisa acontece quando uma página é nova. Denominamos referência de página, a requisição, com o acesso aos seus dados para processamento, de uma página, esteja ela ou não em memória.

As páginas menos recentes tendem a estar no final da lista (LRU), então quando o *Buffer Pool* está cheio e não tem memória suficiente para alocar novas páginas, podemos optar por remover a página menos recente (estratégia LRU) ou a página mais recente (estratégia MRU) que está no topo da lista.

Least Frequently Used (LFU) é uma política de substituição que mantém um contador de referências para cada página. Este contador indicará qual frequência esta página é acessada, incrementando o contador a cada nova referência. A página com o menor contador (página menos frequente) será escolhida como vítima. Existem diversas variações desta política, e ela pode ser

implementada de várias maneiras, por exemplo, com uma lista ordenada pela frequência de cada página. A LFU apresenta algumas desvantagens, como por exemplo, a complexidade de manter a ordenação das páginas e também o fato que o algoritmo poder preservar uma página frequente por muito tempo mesmo que essa página não tenha sido referenciada posteriormente, devido à páginas com os contadores com uma alto valor. Para mitigar este problema, existem estratégias para envelhecer as páginas (*aging*). Por exemplo, *Least Frequently Used with Dynamic Aging* (LFU-DA) (ARLITT *et al.*, 2000) usa o fator de inflação L inicializado em 0. O valor de L é sempre atualizado com o valor do contador da última vítima. Quando uma nova página é inserida, o contador de frequência é adicionado com o valor atual de L , diminuindo o esforço para que as novas páginas possam aumentar a sua frequência antes de serem selecionadas como vítimas.

First in First out (FIFO) é uma política de substituição simples que usa uma fila de páginas. A inserção de novas páginas acontece no início da fila. Quando uma substituição é necessária, a página no final da fila é removida. Ao contrário de LRU e MRU, o algoritmo FIFO não move nenhuma página quando elas são referenciadas novamente.

2.3 Problemas Inerentes ao Gerenciamento de *buffer*

Adicionalmente aos mecanismos básicos de gerenciamento de *buffer* visto na seção anterior, existem problemas inerentes ao processo de gerenciamento de *buffer*. Os métodos de gerenciamento de *buffer* (políticas de substituição) podem endereçar total ou parcialmente estes problemas.

2.3.1 Operações Sequenciais e *Scan resistance*

Um dos recursos essenciais de uma política de substituição sofisticada é o *scan resistance*, que coíbe que o *buffer* seja todo poluído com páginas recentes, possivelmente, removendo páginas mais referenciadas.

É uma operação relativamente comum quando o processador de consultas executa a operação sequencial (por exemplo, uma varredura completa na tabela ou índice – *sequential scan*). Várias dessas páginas recentes possivelmente nunca serão referenciadas novamente durante a execução da aplicação, e talvez nem mesmo depois, em futuro próximo. Operações sequenciais podem remover páginas frequentes ou importantes dentro do contexto de execução do gerenciador de *buffer* artificializando o reuso das páginas. Algoritmos como o ARC (ver

Seção 3.1.9) (MEGIDDO; MODHA, 2003) usam mecanismos de *scan resistance* para reduzir o impacto dessas operações sequenciais.

SGBS comerciais vem tentando minimizar a quantidade de dados que são trazidos para o *Buffer Pool* e dificilmente serão acessados, o objetivo é manter as páginas acessadas com uma maior frequência no *Buffer Pool* mesmo que um longo acesso sequencial seja executado. Para isso, SGBDs como o ORACLE e MYSQL optam em utilizar uma estratégia de inserção de páginas recentes mais sofisticada (ver Seção 3.1.3).

2.3.2 *Cache Starvation*

Várias políticas de substituição dividem o *buffer* em regiões (geralmente listas) com tamanhos fixos ou dinâmicos. Cada região pode manter páginas recentes, frequentes, *clean* ou *dirty*, ou combinações desses tipos. Essas regiões competem por uma fração do *buffer* e dependendo da carga de trabalho, uma dessas regiões será favorecida e seu tamanho será mais significativo. Consequentemente, as outras regiões se tornarão menores.

O problema de *cache starvation* ocorre quando uma região se torna muito pequena e os mecanismos da política de substituição impedem que ela cresça novamente, possivelmente levando à degradação do desempenho do algoritmo. Evitar o *cache starvation* é uma qualidade essencial no desenvolvimento de novas políticas de substituição.

2.3.3 *Páginas fantasmas (ghost)*

No projeto de algoritmos de substituição, quando uma página é removida do *buffer*, o espaço reservado para dados é removido ou substituído da memória dependendo da implementação. No entanto, uma abordagem comum é manter apenas alguns metadados dessa página despejada na memória, por exemplo, o identificador de página. Assim, o algoritmo pode fazer suposições sobre essa página despejada caso ela seja acessada novamente no futuro. O algoritmo 2Q (ver Seção 3.1.5) usa esta abordagem para identificar *hot pages*, que são páginas consideradas importantes para o algoritmo de substituição durante o seu contexto de execução (SHASHA; JOHNSON, 1994).

O algoritmo LIRS (ver Seção 3.1.8), publicado anos depois de 3.1.5, usou essa abordagem de manter *ghost pages* chamando pelo termo *non-resident pages*. Posteriormente, o algoritmo ARC (ver Seção 3.1.9) usou o termo *ghost caches*. Neste trabalho, usaremos o termo *ghost pages* para explicar os algoritmos.

2.4 Mídia de armazenamento assimétrica

Vários algoritmos de substituição foram desenvolvidos ao longo dos anos considerando aspectos de escolha de vítima como a frequência, recência e falhas de página (*miss*). Porém, com a evolução das novas tecnologias de armazenamento e a popularização dos SSDs, um novo aspecto surgiu e teve que ser considerado no desenvolvimento de políticas de substituição de páginas sofisticadas.

As unidades de disco rígido HDD exigem o mesmo tempo e custo para operações de leitura e gravação. Portanto, o acesso aos dados é feito de forma simétrica. Por outro lado, os SSDs possuem um acesso à mídia de forma assimétrica. O tempo e o custo das operações de gravação em um SSD são mais significativos do que as operações de leitura (PARK *et al.*, 2011).

Consequentemente, para que uma estratégia de substituição se beneficie da assimetria, ela deve ter mecanismos para diferenciar as páginas entre leitura e escrita. Se o algoritmo conseguir reduzir o número de páginas de escrita despejadas, é esperado que o tempo e o custo das transações serão menores, melhorando assim o desempenho.

O algoritmo *Clean-First LRU* (ver Seção 3.2.1) foi a primeira estratégia desenvolvida para trabalhar com SSD (memória flash). Várias novas estratégias foram desenvolvidas nos anos seguintes motivadas pela CFLRU e novas mídias de armazenamento assimétricas também surgiram. Portanto, tirar proveito da assimetria da mídia de armazenamento é um recurso essencial no desenvolvimento de novas políticas de substituição.

2.5 Algoritmo MIN

O algoritmo MIN é uma estratégia offline, pois a política não é executada em um sistema real de produção, pois é necessário conhecimento futuro da carga de trabalho para a sua execução. A implementação pode usar uma lista de páginas residentes na memória e uma fila com todas as referências de página para o futuro, a ideia básica por trás do processo de substituição de página é escolher uma vítima na lista com menor frequência de uso no futuro (Belady, 1966).

MIN é considerado um algoritmo ótimo e pode maximizar a taxa de acerto (*hit ratio*) de um *buffer* com um número fixo de páginas. Por ser um algoritmo offline, ele é usado para realizar testes de desempenho e comparação empírica para encontrar a melhor taxa de acertos para uma determinada carga de trabalho.

2.6 Conclusão

Este capítulo reviu a arquitetura e problemas enfrentados pelo gerenciador de *buffer*, em especial na sua política de substituição de páginas. Estudou também o impacto de novas mídias assimétricas de armazenamento no gerenciamento de *buffer* em SGBDs.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta os trabalhos relacionados estudados durante a revisão bibliográfica. Eles foram classificados em duas categorias. A primeira está descrita na Seção 3.1 que apresenta as políticas de substituição de *buffer* do tipo não assimétricas. A segunda na Seção 3.2 com as políticas de substituição de *buffer* assimétricas. Em seguida, a Seção 3.3 faz uma breve compilação no formato de uma linha do tempo com o objetivo de revelar possíveis relações entre trabalhos apresentados. Por fim, a Seção 3.4, faz uma análise qualitativa e comparativa dos trabalhos relacionados discutidos nas Seções 3.1 e 3.2.

3.1 Políticas de substituição de *buffer* não assimétricas

Esta seção apresenta as políticas de substituição de *buffer* projetadas para mídias não assimétricas. Foram analisados trabalhos relevantes na literatura e identificamos seus potenciais problemas. Essas políticas focam na melhoria da taxa de acertos, embora não considerem o tipo de operação (leitura/gravação). Mecanismos de histórico e envelhecimento sofisticados têm sido propostos. Estes conceitos melhoram a classificação das páginas frequentes e se adaptam melhor aos padrões de acesso.

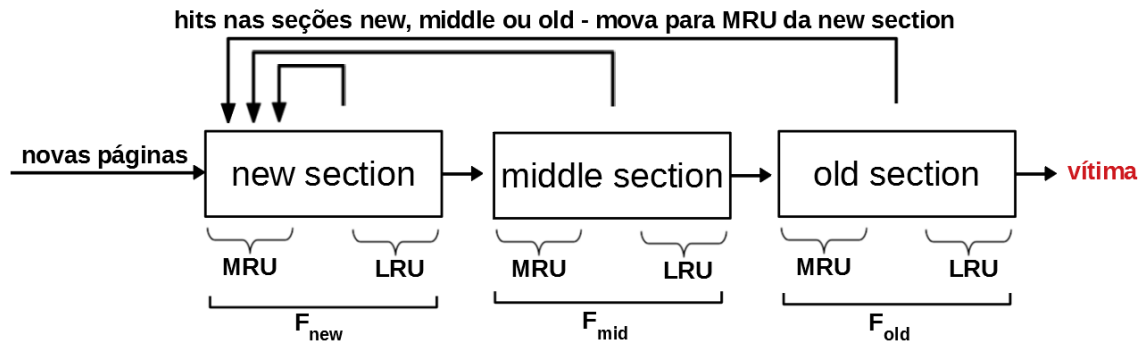
3.1.1 FBR

Frequency-Based Replacement (FBR) foi um dos primeiros algoritmos a combinar aspectos de recência e frequência usando os algoritmos LRU e LFU como base. O FBR divide o *buffer* em três seções: *new*, *middle*, *old*. Cada página tem um contador de frequência semelhante ao LFU. O FBR tem dois parâmetros de ajuste: A_{max} e C_{max} . O parâmetro A_{max} é usado para evitar que os contadores de referência cresçam indefinidamente e o C_{max} é o valor máximo de entidades usadas na estratégia de busca de vítimas (ROBINSON; DEVARAKONDA, 1990).

A Figura 6 descreve o fluxo de execução simplificado do FBR. Os parâmetros de ajuste F_{new} e F_{old} definem o tamanho das seções (F_{mid} é o resto). As novas páginas são inseridas na MRU da seção *new* com a frequência igual a 1. Se a seção *new* estiver cheia, a página LRU da seção *new* é movida para a seção *middle*. Se a seção *middle* também estiver cheia, a página LRU da seção *middle* então será movida para a seção *old*. E, por último, se a seção *old* estiver cheia, o algoritmo escolhe a página com a menor contagem de referência da seção *old* como vítima.

Quando ocorre um *hit*, as páginas são movidas para a MRU da seção *new* e o contador

Figura 6 – FBR fluxo de execução



Fonte: O Autor.

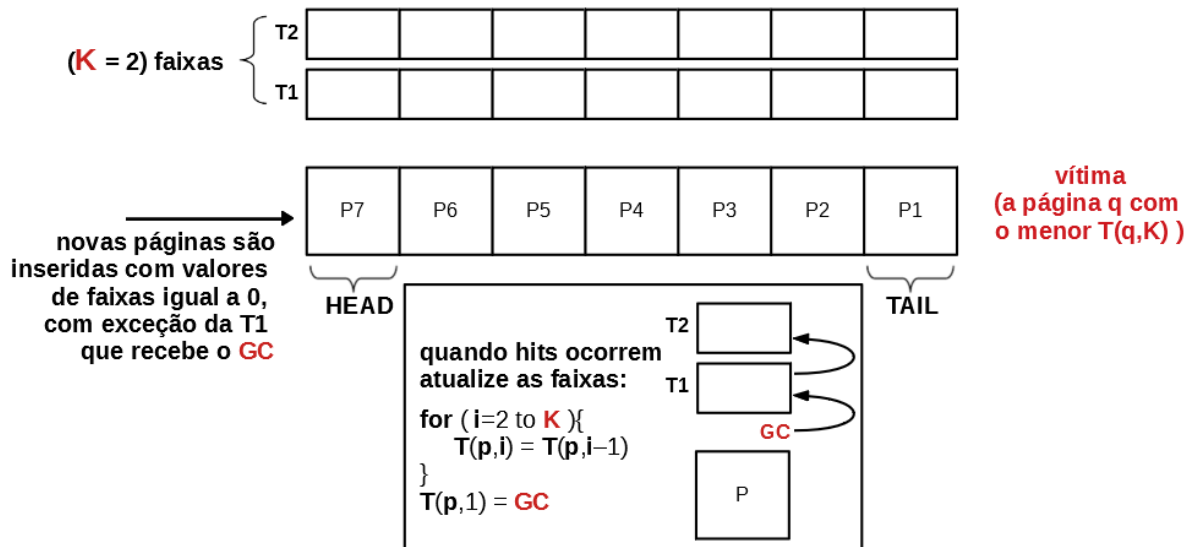
de frequência da página é incrementado. No entanto, o contador não será incrementado se a página já residia na seção *new*. Essa estratégia evita que páginas acessadas com frequência em curtos períodos tenham contadores de referência muito grandes.

A escolha dos tamanhos das seções do FBR influencia na sua performance. Quanto maior o tamanho da seção *new*, maior será o número de páginas recentes e maior será o tempo de vida em que uma página reside no *buffer*, pois devem primeiramente envelhecerem até serem deslocadas para as nas seções *middle* e *old*. Quanto maior o tamanho da seção *middle*, maior será a probabilidade de uma página ser referenciada novamente sem correr o risco no pior caso (*miss*) de ter sido movida para a seção *old* e selecionada como vítima anteriormente. Quanto maior o tamanho da seção *old*, durante a escolha da vítima mais páginas com diferentes contadores de referência podem ser consultados.

3.1.2 LRU-K

O algoritmo LRU não discrimina com sucesso a frequência das páginas, pois só pode considera a última referência por página. O algoritmo LFU discrimina a frequência, mas tem a desvantagem de manter as páginas com alta frequência por muito tempo no *buffer* mesmo que não sejam novamente referenciadas. Portanto, é necessário uma estratégia de envelhecimento, por exemplo, decrementos de contador de referência. O *Least kth-to-last Reference* (LRU-K) (O'NEIL *et al.*, 1993) melhora o algoritmo LRU mantendo um histórico de faixas K , com o tempo das últimas K referências relacionadas a cada página no *buffer*. O K representa um parâmetro de ajuste. Quando K é igual a 1 (LRU-1), o algoritmo funciona como um LRU simples. Ao contrário do LFU, o algoritmo aplica uma estratégia sofisticada de envelhecimento sem ter que diminuir constantemente os contadores de referência.

Figura 7 – LRU-K fluxo de execução



Fonte: O Autor.

A Figura 7 descreve o fluxo de execução simplificado do LRU-K. O algoritmo usa um contador global (GC) que é incrementado para cada solicitação e outro parâmetro de ajuste chamado *Correlated Reference Period* (CRP). CRP é uma estratégia otimista que mantém a página no *buffer* por um curto período, evitando que ela seja imediatamente escolhida como vítima.

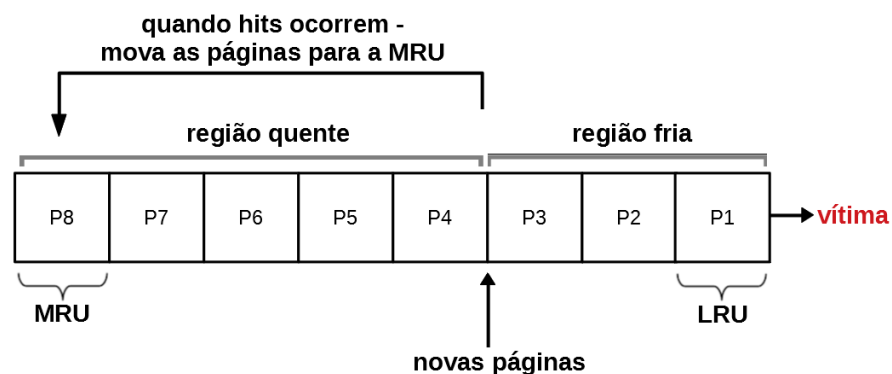
Neste exemplo, vamos supor que CRP é igual a 0 e K é igual a 2 (LRU-2), para simplificar a explicação do algoritmo. Novas páginas são inseridas em uma fila de prioridade e cada faixa associada a essa nova página é definida como 0, exceto a faixa 1 ($T1$), que é definida como o valor atual do contador global. Quando ocorre um *hit*, os valores de cada faixa devem ser atualizados. Em primeiro lugar, os valores das faixas da página referenciada são deslocados. Neste caso (LRU-2), a última referência em $T1$ é definida como $T2$. Finalmente, $T1$ é definido como o valor atual do GC .

A escolha da vítima é feita selecionando uma página com o menor valor na faixa K . Neste caso (LRU-2), a página com o menor valor na faixa $T2$. Para encontrar a vítima com o menor valor, o LRU-K usa uma fila de prioridade, que pode ser implementada com uma estrutura de heap, por exemplo. Assim, a complexidade do algoritmo é $O(\log N)$. Como as faixas de uma página do histórico são atualizadas quando ocorrem *hits*, as referências mais antigas são descartadas e consequentemente não serão mais usadas para selecionar uma vítima isso permite que páginas anteriormente consideradas frequentes sejam envelhecidas a medida que a carga de trabalho muda.

3.1.3 LRU-MIS (LRU with Midpoint Insertion Strategy)

O algoritmo LRU simples não possui *scan resistance* (ver Seção 2.3.1). Uma possível solução para essa vulnerabilidade é a estratégia de inserção de ponto médio. A Figura 8 descreve o fluxo de execução do LRU-MIS. A diferença para a LRU simples é que as novas páginas são inseridas em um ponto médio na lista, dividindo a LRU em duas sublistas, regiões quentes e frias. Portanto, as páginas são inseridas no lado da MRU somente quando ocorre um *hit*. A escolha da vítima é a mesma do algoritmo LRU simples.

Figura 8 – LRU-MIS execution flow



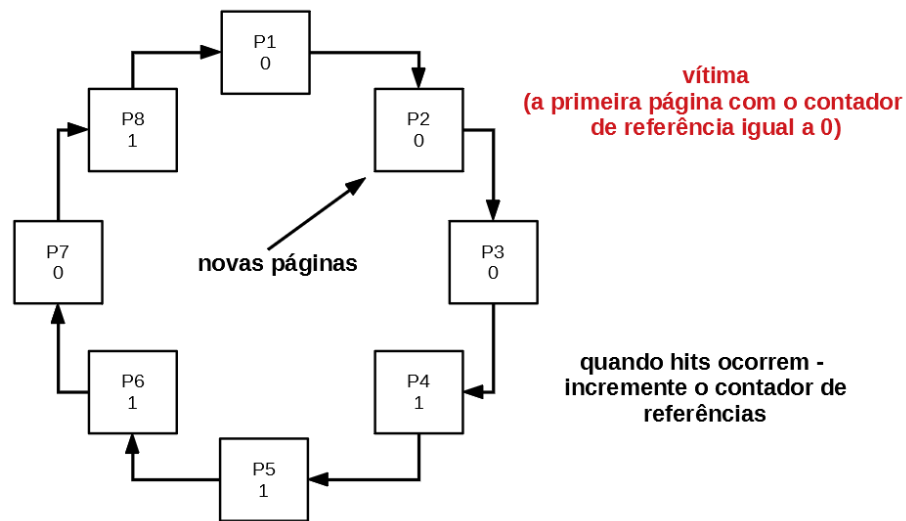
Fonte: O Autor.

A inserção de novas páginas no ponto médio evita que elas eventualmente poluam o *buffer* em caso de grandes varreduras sequenciais, preservando assim as páginas usadas mais recentemente que já foram referenciadas mais de uma vez. A localização do ponto médio é um parâmetro de ajuste. Os SGBDs comerciais ORACLE e MySQL usam estratégias semelhantes de inserção (ORACLE, 2021b) (ORACLE, 2021a). No caso do MySQL, é usado 62,5% (5/8) do *buffer* para páginas na região quente.

3.1.4 GCLOCK (Generalized Clock)

Clock, também conhecido como *Second Chance*, é um algoritmo de substituição destinado a dar às páginas uma segunda chance no *buffer*. O nome *Clock* é devido à sua implementação usando uma lista circular de páginas com um ponteiro de página (CORBATÓ; TECHNOLOGY, 1968). GCLOCK é apenas uma generalização de *Clock*, dos quais os bits de referência não estão limitados a apenas 1 ou 0. GCLOCK usa um contador e dois parâmetros de ajuste: contador inicial (*IC*) e contador de *hits* (*HC*) (NICOLA *et al.*, 1992).

Figura 9 – GCLOCK fluxo de execução



Fonte: O Autor.

A Figura 9 descreve o fluxo de execução do GCLOCK. Cada página está associada a um contador de referência, semelhante à política LFU. Novas páginas são inseridas usando um ponteiro em uma lista circular com um contador de referência igual a IC . Quando ocorre um acerto, este contador é definido para o valor do HC . Quando o *buffer* está cheio, o ponteiro é consultado; se a contagem de referência for igual a 0, essa página é escolhida como vítima, a nova página é inserida nesta posição atual e o ponteiro avança para a próxima posição. Caso contrário, o algoritmo decrementa o contador de referência em 1 e o ponteiro avança para a próxima posição. Este processo é repetido até que seja encontrada uma página com o contador de referência igual a 0.

O SGBD comercial PostgreSQL implementa uma variante chamada *Clock Sweep* (XIA; BU, 2015) que considera tanto o contador de referência, mas também um sinalizador, que indica se uma transação estiver acessando esta página. Uma vantagem é que se a vítima for acessada por uma transação e não puder ser despejada, é bem simples escolher outra vítima avançando o ponteiro Clock. A desvantagem do Clock é que quando o ponteiro gira várias vezes procurando uma vítima pode dificultar a execução das transações.

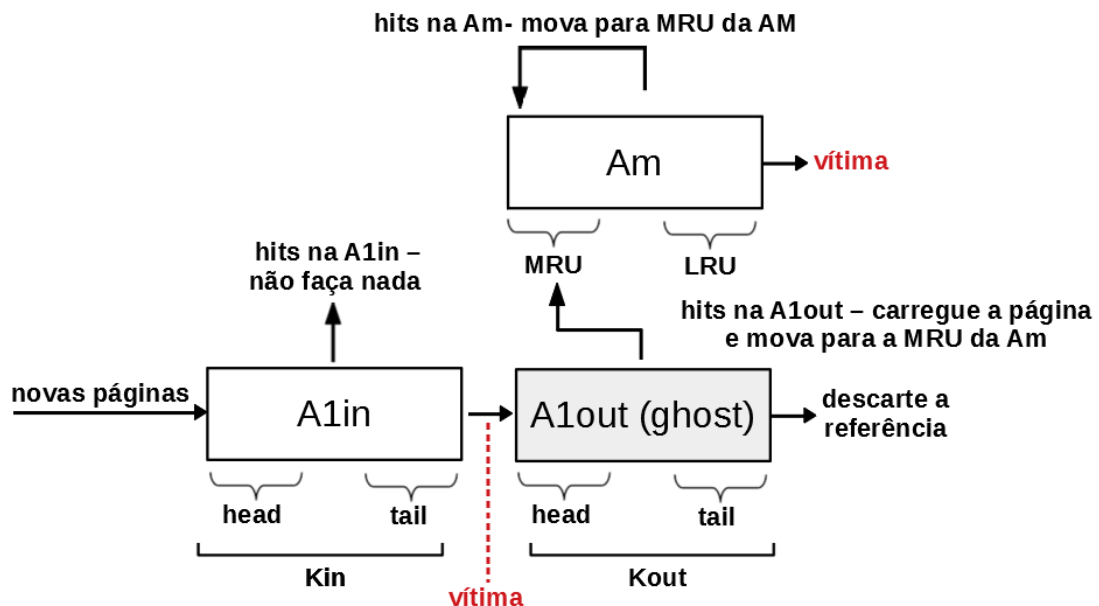
3.1.5 2Q

Uma das motivações para a criação do algoritmo 2Q foi superar o algoritmo LRU-K, o problema com o LRU-K é que cada solicitação de página requer complexidade em $O(\log N)$, onde N representa o tamanho do *buffer*. Assim, o 2Q foi criado com o objetivo de ter um baixo

overhead com complexidade constante $O(1)$ e desempenho em todas as requisições. O 2Q tem duas variações: a versão simplificada e a versão completa. Este trabalho descreverá a versão completa, que é composta por três filas, chamadas A1in, Am e A1ou (SHASHA; JOHNSON, 1994).

- A1in: Controla a recência das páginas usando o algoritmo FIFO.
- Am: Controla a frequência das páginas usando o algoritmo LRU.
- A1out: Recebe as páginas despejadas do A1in e determina quais páginas inserir no Am. A1out é uma lista *ghost*, pois os dados nas páginas não são mantidos na memória, apenas metadados, como os identificadores de páginas.

Figura 10 – 2Q fluxo de execução



Fonte: O Autor.

A Figura 10 descreve o fluxo de execução do 2Q. Em caso de *miss*, as novas páginas são carregadas na memória e inseridas na *head* de A1in. Quando ocorre *hits* em A1in nada é feito, porque as páginas com muitas referências próximas não são imediatamente consideradas páginas frequentes, as páginas são envelhecidas. *Hits* em Am significa que as páginas são movidas para o Am MRU. *Hits* em A1out significam que as páginas são carregadas na memória e inseridas no Am MRU.

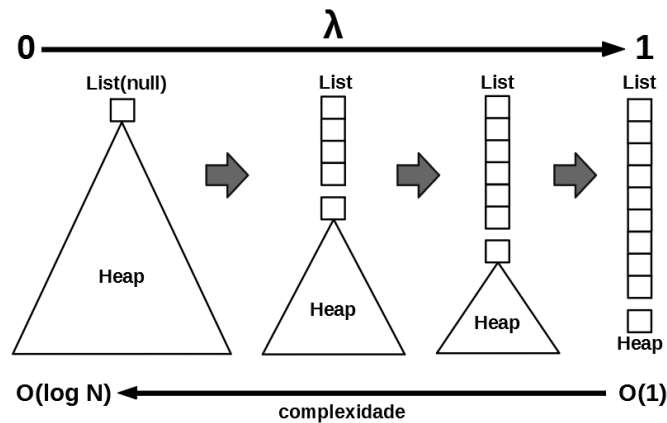
O espaço real do *buffer* é dividido entre A1in e Am (porque A1out é uma lista *ghost*). O *Kin* e *Kout* são parâmetros de ajuste que representam o tamanho máximo de crescimento de A1in e A1out, respectivamente. Quando o *buffer* está cheio e é necessária uma substituição, uma vítima é escolhida entre A1in ou Am, seguindo esta condição: se o tamanho atual de A1in

for maior que K_{in} , significa que A_{lin} excedeu o tamanho máximo e a página final de A_{lin} é despejada como vítima. Caso contrário, a página LRU de A_m é escolhida como vítima. Todas as páginas despejadas do A_{lin} são inseridas no A_{lout} como *ghosts*. Se o tamanho atual de A_{lout} for maior que K_{out} , a página do final de A_{lout} (*tail*) será descartada.

3.1.6 LRFU

Lidar com recência e frequência é um desafio na implementação de políticas de substituição. *Least Recently/Frequently Used* (LRFU) é uma política que assume um comportamento de escolha de vítimas entre LRU e LFU usando um parâmetro de ajuste λ . Em outras palavras, quando o valor de λ é igual ou próximo de 0, o LRFU tende a funcionar de forma igual ou semelhante ao LFU. Quando o valor de λ se aproxima de 1, a política LRFU tende a funcionar como uma política baseada em recência (KIM *et al.*, 1996).

Figura 11 – LRFU - impacto do λ nas estruturas de dados



Fonte: O Autor.

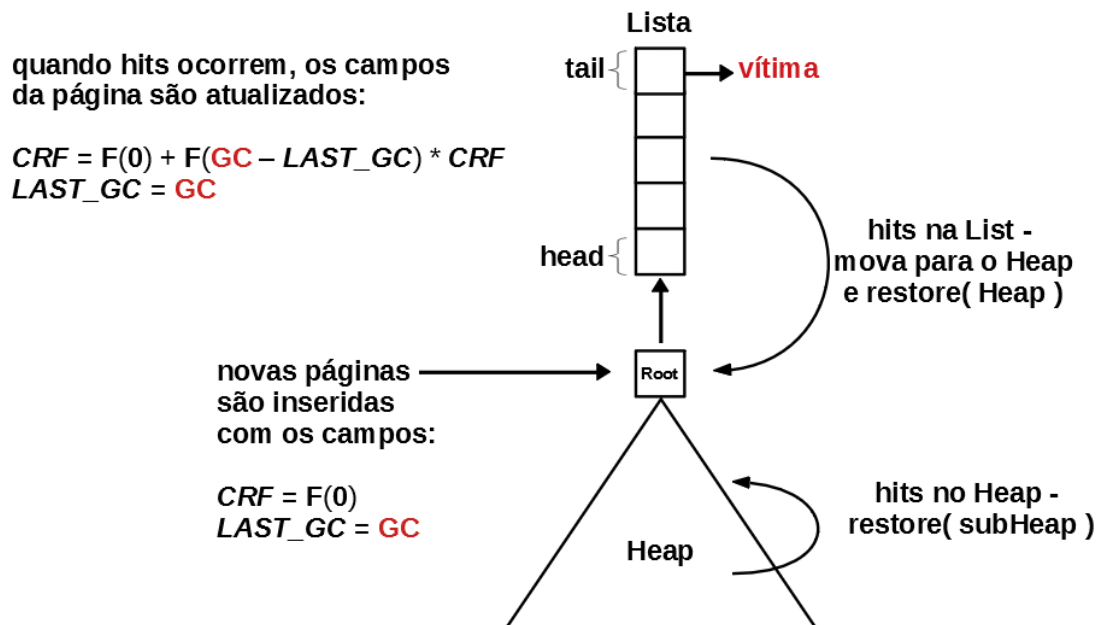
A LRFU pode ser implementada usando duas estruturas de dados, uma lista e um *heap*. O parâmetro λ influencia diretamente no tamanho dessas estruturas, conforme mostrado na Figura 11. Assim, a complexidade computacional do algoritmo também depende do valor de λ , variando entre a complexidade das políticas LRU e LFU, $O(1)$ e $O(\log N)$, respectivamente.

Na política LRFU, cada solicitação de página incrementa um contador global (GC). Cada página tem dois campos. O primeiro é $LAST_GC$, é um campo que armazena o valor do contador global do momento em que a página foi referenciada pela última vez. O segundo campo, chamado *Combined Recency and Frequency* (CRF), determina a importância dessa página segundo o algoritmo. CRF é calculado com uma função de peso, usando a Fórmula 3.1, onde x é a distância de referências entre GC e $LAST_GC$ daquela página específica.

$$F(x) = \frac{1}{2}^{\lambda x} \quad (3.1)$$

A Figura 12 descreve o fluxo de execução simplificado do LRFU. A estrutura do *heap* é organizada com o valor CRF de cada página. As novas páginas são inseridas na estrutura *heap* com as informações CRF e *LAST_GC*. Quando o *buffer* está cheio, a página antiga na raiz do *heap* é inserida no início da lista (*head*). Em seguida, a página no final da lista (*tail*) é selecionada como vítima, pois o LRFU procura substituir as páginas com o menor CRF.

Figura 12 – LRFU fluxo de execução



Fonte: O Autor.

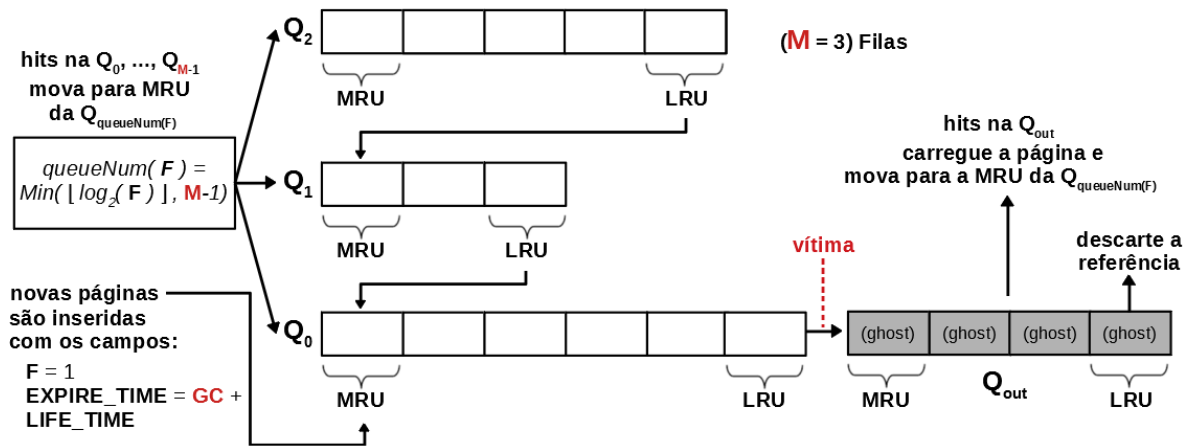
Sempre que ocorre um *hit*, os valores CRF e *LAST_GC* são atualizados. Se a página estiver na lista ela será promovida e inserida no *heap* e a estrutura será restaurada chamando a função *restore()* para o *heap*. Se a ocorrência ocorrer para uma página já dentro do *heap*, só será necessário restaurar uma parte do *heap* (*subheap*) referente essa página.

O LRFU possui uma variante chamada Window LRFU (BAI *et al.*, 2016), que busca se adaptar melhor as mudanças de padrões de acesso usando uma estratégia de janela deslizante que mantém um histórico das últimas requisições.

3.1.7 MQ

O algoritmo *Multi-Queue* (MQ) apresenta um *buffer* com M filas LRU denominadas $Q_0, \dots, Q_{M-1}$ com tamanhos dinâmicos ajustados durante a execução e uma fila *ghost* Q_{out} com um tamanho fixo semelhante ao 2Q (ver Seção 3.1.5). O algoritmo prioriza páginas frequentes, mas ao mesmo tempo, aplica mecanismos de envelhecimento usando as filas como um ranking de páginas. M , Q_{out} e $LIFE_TIME$ são parâmetros de ajuste. $LIFE_TIME$ indica o tempo de vida da página. O MQ mantém um contador global de referências (GC) e cada página possui um contador de referências (F), e um campo chamado $EXPIRE_TIME$ usado para envelhecimento, calculado pela soma do GC atual e $LIFE_TIME$ (ZHOU *et al.*, 2001).

Figura 13 – MQ fluxo de execução



Fonte: O Autor.

A Figura 13 descreve o fluxo de execução do MQ. Novas páginas são inseridas no MRU de Q_0 com seu contador de frequência igual a 1 e o $EXPIRE_TIME$ calculado. Quando ocorre um *hit* em uma página que reside em uma das filas M , o campo F é incrementado e $EXPIRE_TIME$ é atualizado com o GC atual. A página é promovida para o lado MRU de uma das filas. O algoritmo usa a função, chamada *queueNum*, que indica em qual das filas M a página deve ser movida.

Quando o *buffer* está cheio, o MQ seleciona LRU da fila Q_0 como vítima. Se esta fila estiver vazia, a próxima LRU da fila superior será selecionada e assim por diante. A página vítima é movida para a fila Q_{out} como uma *ghost*, preservando seus metadados, como o campo F . Se a fila Q_{out} estiver cheia, a referência é finalmente descartada. Quando ocorre um *hit* na fila Q_{out} , a página é carregada no *buffer* e todos os campos são atualizados e a página é movida para uma das filas M usando a função *queueNum*.

No processo de envelhecimento, o algoritmo MQ verifica em cada solicitação as páginas LRU de cada uma das filas M (exceto Q_0). Se $EXPIRE_TIME < GC$, a página é rebaixada para MRU da fila inferior, impedindo que esta página frequente resida no *buffer* por um período prolongado.

O MQ tem uma complexidade constante e usa um mecanismo de envelhecimento sem a necessidade de decrementar contadores. Além disso, quanto maior o número de filas, mais páginas podem ser analisadas pelo mecanismo de envelhecimento. No entanto, um número muito alto de filas pode afetar o tempo de execução e o consumo de memória.

3.1.8 LIRS

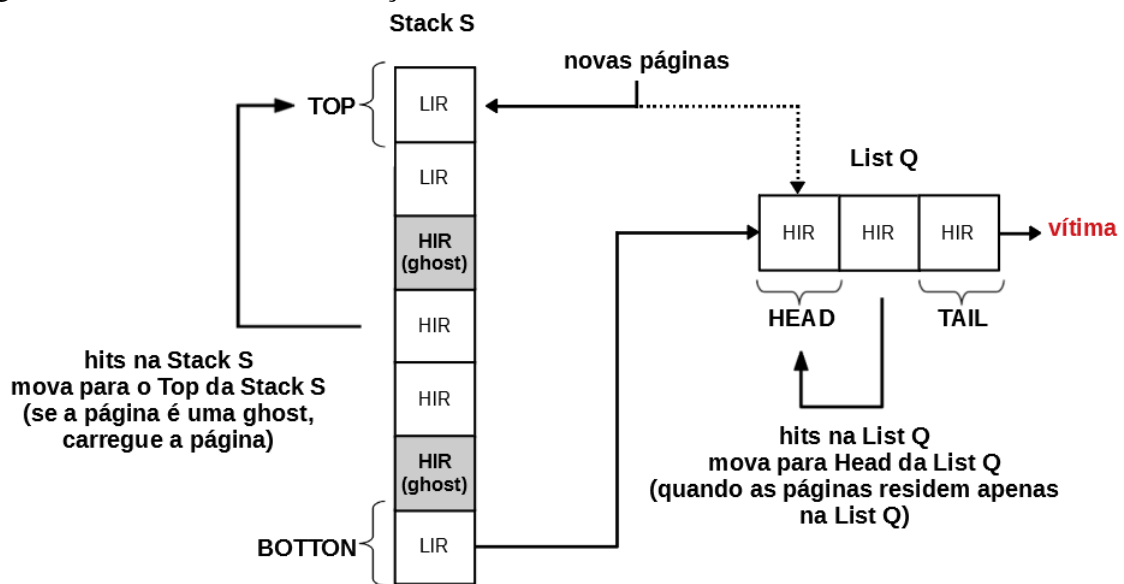
O *Low Inter-reference Recency Set* (LIRS) foi baseado no princípio *Inter-Reference Recency* (IRR). Considerando uma página P , o valor $IRR(P)$ pode ser calculado como o número de acessos distintos de outras páginas entre dois acessos consecutivos de P , por exemplo, o último e o penúltimo acesso de P . Portanto, com este princípio, as páginas com um IRR baixo tendem a ser re-acessadas segundo o algoritmo (JIANG; ZHANG, 2002).

O LIRS classifica as páginas em dois tipos de status, LIR (*Low IRR*) e HIR (*High IRR*). O objetivo é eliminar as páginas HIR do *buffer* primeiro. O tamanho do *buffer* em páginas é L , o algoritmo mantém um número máximo de páginas LIRs (*LIR size*) para as páginas classificadas como LIR e HIRs (*HIR size*) para as páginas classificadas como HIR, onde $L = LIRs + HIRs$. Esses dois tamanhos são parâmetros de ajuste. Os autores descrevem que o HIRs pode ter um tamanho pequeno, por exemplo, 1% de L .

A implementação do LIRS é baseada no algoritmo LRU e usa uma pilha S e uma lista Q . A Figura 14 descreve o fluxo de execução do LIRS. Novas páginas são inseridas no topo da pilha S com status LIR. Quando o LIRs (número de páginas LIR) é alcançado, as novas páginas são inseridas com status HIR no topo da pilha S e no topo da lista Q simultaneamente. Isso não significa que a página será duplicada porque as estruturas podem usar ponteiros para o mesmo elemento. Assim, uma página pode residir tanto na pilha S quanto na lista Q . No entanto, apenas páginas com status HIR podem residir na lista Q .

Quando novas páginas são inseridas e o *buffer* está cheio, os tamanhos LIRs e HIRs são alcançados e a estratégia escolhe a página final da lista Q (*tail*) como vítima. Se esta página também residir na pilha S , o algoritmo mantém apenas os metadados, convertendo-os em uma página HIR ghost. Existem quatro casos possíveis ao solicitar páginas que já residem no *buffer*

Figura 14 – LIRS fluxo de execução



Fonte: O Autor.

ou são páginas ghost:

- Caso 1: *Hits* em páginas LIR na pilha S, a página é movida para o topo da pilha S, de forma semelhante ao algoritmo LRU. Se esta página estiver na parte inferior da pilha S, a operação de remoção da pilha será executada. Essa operação garante que apenas as páginas com status LIR residam na parte inferior da pilha S. Todas as páginas com status HIR acima da parte inferior da pilha S são removidas até que uma nova página LIR seja encontrada.
- Caso 2: *Hits* em páginas HIR na pilha S, a página é movida para o topo da pilha S e é promovida ao status LIR, o algoritmo também remove sua cópia da lista Q. Além disso, é necessário rebaixar a página LIR da parte inferior da pilha S para o status HIR. Se a página rebaixada também residir na lista Q, o algoritmo também removerá essa cópia. Em seguida, a página rebaixada é inserida no *head* da lista Q e a operação de remoção de pilha é executada.
- Caso 3: *Hits* em páginas *ghost* HIR na pilha S são considerados uma falha (*miss*) porque os dados da página não residem no *buffer*. Antes que os dados da página possam ser recarregados, é necessário escolher uma vítima para liberar espaço. Então, o processo é semelhante ao Caso 2, mas sem remover uma cópia da lista Q.
- Caso 4: *Hits* na página HIR na lista Q e que não residem na pilha S. Uma nova entidade de página é inserida no topo da pilha S. O status HIR da página é mantido e a página é movida para *head* da lista Q.

Uma das limitações do algoritmo LIRS é a operação de remoção de pilha. Dependendo da carga de trabalho, a estratégia pode ter um grande número de páginas *ghost* HIR residindo na pilha S. Isso pode afetar a complexidade do algoritmo, que não será constante em cenários específicos.

CLOCK-Pro (JIANG *et al.*, 2005) combina o algoritmo *Clock* (ver Seção 3.1.4) com LIRS. O *Clock* agora tem páginas *ghost* e usa novos mecanismos para diferenciar entre páginas quentes e frias. Anos depois, o CLOCK-Pro+ (LI, 2019) combinou o antigo CLOCK-Pro com o algoritmo CAR (ver Seção 3.1.10) e propôs uma nova abordagem para se adaptar a diferentes padrões de cargas de trabalho.

Outra variante é o *Dynamic LIRS* (DLIRS) (LI, 2018), que combina os mecanismos de adaptação do ARC (ver Seção 3.1.9) com o LIRS. O DLIRS controla dinamicamente o número de páginas HIR ou LIR no *buffer*.

As variantes *Probability-based Adjustable algorithm on Low Inter-reference Recency Set* (PA-LIRS) (WANG *et al.*, 2020) e LIRS-WSR (JUNG *et al.*, 2007) combinam LIRS com mecanismos semelhantes a LRU-WSR (ver Seção 3.2.2), reduzindo o número de gravações e aproveitando a mídia assimétrica. A diferença é que o PA-LIRS usa mecanismos para ajustar o número de páginas HIR e LIR dinamicamente.

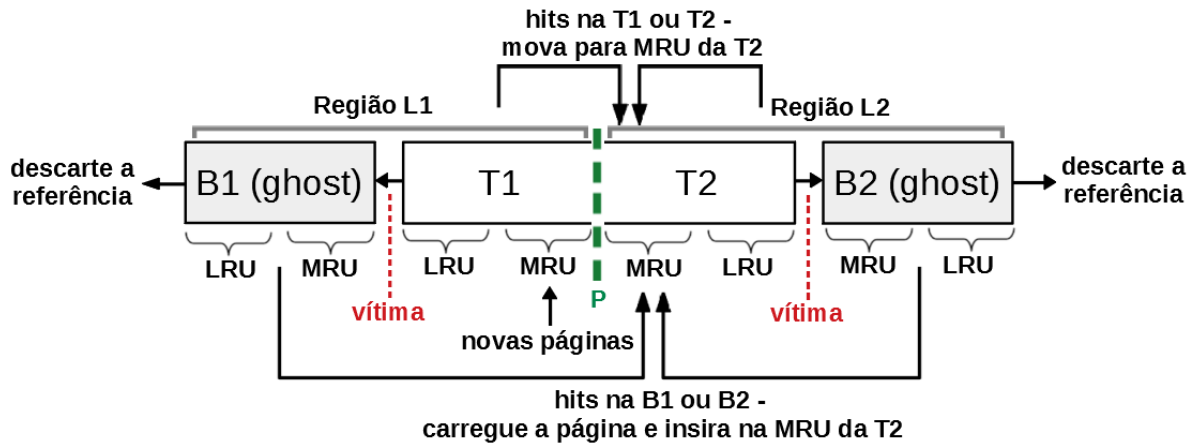
3.1.9 ARC

O *Adaptive Replacement Cache* (ARC) foi projetado para dar suporte as mudanças de padrões das cargas de trabalho. O algoritmo equilibra a frequência e recência ao se adaptar dinamicamente e melhorar o *hit ratio* com uma complexidade constante. ARC é composto por duas regiões, L1 e L2, cada uma com duas LRUs. L1 é dividido em T1 e B1, e L2 é dividido em T2 e B2. A soma dos tamanhos de T1 e T2 deve ser $\leq C$, onde C representa a capacidade total do *buffer* (MEGIDDO; MODHA, 2003).

- T1: Controla a recência das páginas usando o algoritmo LRU.
- T2: Controla a frequência das páginas usando o algoritmo LRU.
- B1: Recebe as páginas despejadas de T1 como *ghosts* e indica o crescimento de T1.
- B2: Recebe as páginas despejadas de T2 como *ghosts* e indica crescimento de T2.

A Figura 15 descreve o fluxo de execução do ARC. Em caso de falha (*miss*), as novas páginas são carregadas na memória e inseridas na MRU de T1. Para *hits* em T1 ou T2 as páginas são movidas para MRU de T2. *Hits* em B1 ou B2, as páginas são carregadas na memória

Figura 15 – ARC fluxo de execução



Fonte: O Autor.

e inseridas na MRU de T2. As vítimas de T1 e T2 vão para B1 e B2 como páginas *ghost*, respectivamente. As vítimas de B1 e B2 são descartadas.

P ($0 \leq P \leq C$) é uma variável que funciona como um *advisor*. Indicando o tamanho desejado para T1 e também é utilizado no momento da substituição. Inicialmente, o tamanho de P é igual a $|T1|$. Hits em B1 são indicativos de que T1 precisa de mais espaço, então P é incrementado seguindo a fórmula: $P = \min(C, P + \max(|B2| / |B1|, 1))$. Esta fórmula considera que se o tamanho de B2 for maior que B1, maior será o incremento de P . A mesma coisa acontece quando ARC tem hits em B2, mas usa a fórmula oposta para diminuir o tamanho: $P = \max(0, P - \max(|B1| / |B2|, 1))$.

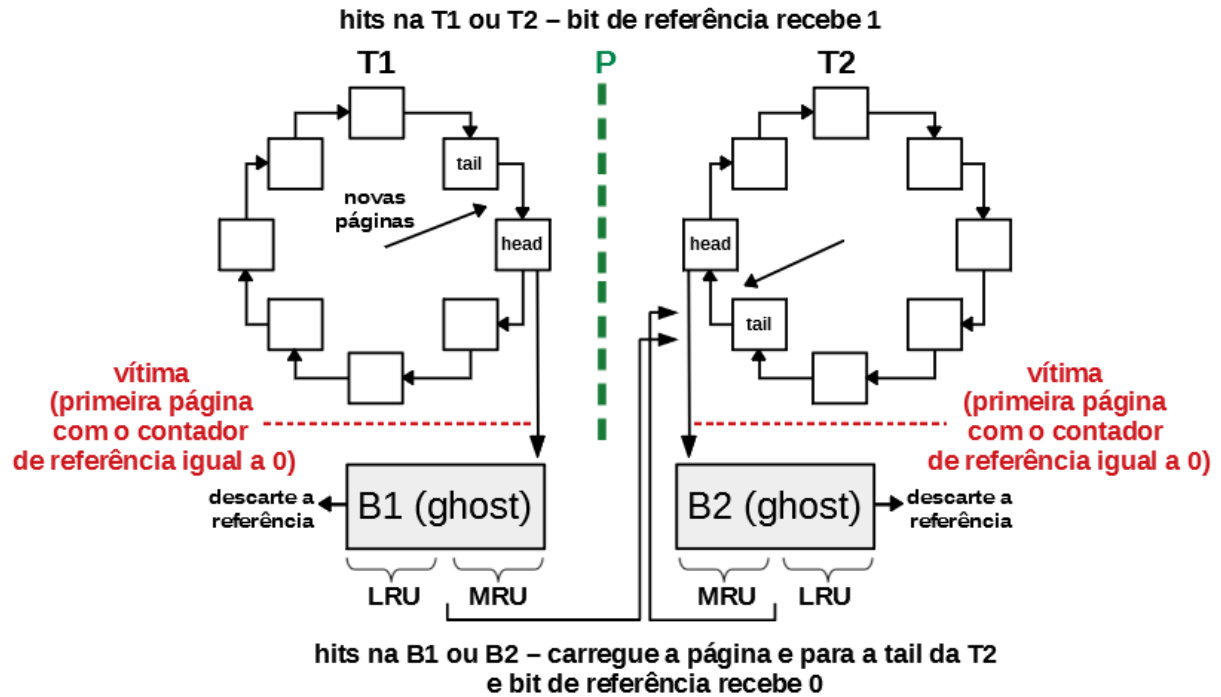
Quando $|T1| + |T2| = C$, o *buffer* está cheio, então o algoritmo deve escolher uma vítima de T1 ou T2 dependendo do valor de P . Grosso modo, se o tamanho atual de T1 for maior que P , o ARC deve escolher a LRU de T1 como vítima, caso contrário a LRU de T2. Em seguida, a vítima será movida para sua respectiva lista *ghost*. O ARC também possui um recurso *scan resistance* (ver Seção 2.3.1). B1 deve aumentar com recebimento de muitas páginas de T1 durante uma longa sequência de páginas recentes, poucos ou nenhum hit ocorrem em B1. Portanto, o algoritmo deve favorecer T2 mantendo páginas frequentes.

3.1.10 CAR

Clock with Adaptive Replacement (CAR) combina as políticas de substituição ARC e *Clock* (ver Seções 3.1.9 e 3.1.4). Uma das vantagens de usar o *Clock* é que durante o processamento das transações, o algoritmo suporta uma melhor concorrência do que o algoritmo LRU. O CAR adapta vários mecanismos ARC, mas mantém o uso de T1 para recência e T2 para

frequência com dois *Clocks*. Cada página agora tem um bit de referência. As listas fantasmas B1 e B2 permanecem as mesmas do ARC (BANSAL; MODHA, 2004).

Figura 16 – CAR fluxo de execução



Fonte: O Autor.

A Figura 16 descreve o fluxo de execução do CAR. As novas páginas são inseridas na *tail* de T1 com o bit de referência igual a 0. Quando ocorrem *hits* em T1 ou T2, apenas o bit de referência é definido como 1. Quando há *hits* em B1 ou B2, o CAR inicialmente escolhe uma vítima, então a página é inserida na *tail* de T2 com o bit de referência igual a 0. O valor do *advisor* **P** é atualizado, semelhante ao ARC.

O CAR usa o *advisor* **P** para decidir uma vítima em T1 ou T2. Se a escolha for T1, o algoritmo seleciona a *head* de T1 como vítima se o bit de referência for igual a 0. Em seguida, a página é movida para a MRU de B1 como uma página *ghost*. Caso contrário, se o bit de referência for igual a 1, a página é movida para a *tail* de T2, e o bit de referência é definido como 0. O processo é repetido, verificando a nova *head* de T1. Se a escolha for T2, o procedimento é o mesmo, mas agora usando B2.

Em vários cenários de carga de trabalho, algumas páginas são referenciadas várias vezes em um curto período de tempo e podem não ser referenciadas no futuro. Esse comportamento faz com que o algoritmo classifique a página como frequente imediatamente. Os autores propõem uma variante chamada CART, que impõe testes mais rigorosos até que uma página seja

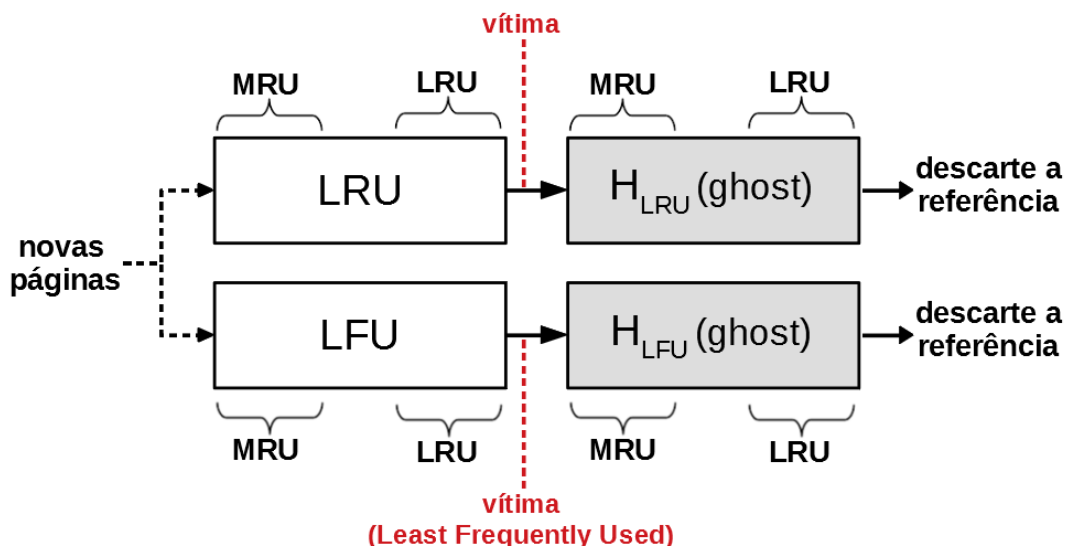
considerada frequente.

CAR e CART não possuem complexidade constante devido ao procedimento de escolha da vítima. No entanto, é possível aproveitar o mecanismo de escolha da vítima com os dois *Clocks* para melhorar a concorrência entre as transações.

3.1.11 LeCaR

Learning Cache Replacement (LeCaR) é uma estratégia de substituição que usa técnicas de aprendizado de máquina. LeCaR usa o conceito de especialistas semelhante ao *Adaptive Caching Using Multiple Experts* (ACME) (ARI *et al.*, 2002). A ACME propõe um conjunto de políticas (especialistas) executadas simultaneamente a cada solicitação. A ACME calcula uma rank para decidir qual é a melhor estratégia no momento com base em suas estatísticas ao recompensar ou punir cada especialista. LeCaR usa *online reinforcement learning* com a técnica de *regret minimization*. Ao contrário do ACME, o LeCaR propõe uma abordagem mais minimalista usando dois especialistas, LRU e LFU, com o suporte de duas listas *ghost*, H_{LRU} e H_{LFU} , e os pesos W_{LRU} e W_{LFU} . Além disso, LeCaR usa dois parâmetros internos, a taxa de aprendizado e a taxa de desconto usada ao executar os mecanismos de aprendizado (VIETRI *et al.*, 2018).

Figura 17 – LeCaR fluxo de execução



Fonte: O Autor.

A Figura 17 descreve o fluxo de execução do LeCaR. Novas páginas são inseridas simultaneamente nas políticas LRU e LFU. Isso não significa que a página será duplicada. O algoritmo pode usar ponteiros nas entidades das estruturas que se referem à mesma página.

Quando ocorre um *hit*, ambas as estratégias atualizam sua estrutura, movendo a página para o lado MRU no caso do LRU e incrementando o contador de referência no caso do LFU. Quando ocorre uma ocorrência em qualquer lista *ghost*, os pesos W_{LRU} ou W_{LFU} são ajustados de acordo com a lista *ghost* que recebeu o *hit*.

O LeCaR primeiro seleciona um dos especialistas como vítima, com base em uma probabilidade aleatória e no valor dos pesos normalizados. Então, se o especialista LFU for escolhido, a página com o menor valor do contador de referência será removida de ambos os especialistas e enviada para a MRU da H_{LFU} como uma *ghost*. Caso contrário, se o especialista LRU for escolhido, a página LRU será removida de ambos os especialistas e movida para H_{LRU} como uma *ghost*.

O algoritmo LFU geralmente é implementado usando um *heap* (ver Seção 3.1.6). Portanto, a complexidade do LeCaR não é constante. O algoritmo CACHEUS (RODRIGUEZ *et al.*, 2021) aprimora o LeCaR usando especialistas mais sofisticados, oferecendo recursos como *scan resistance* (ver Seção 2.3.1) e *churn resistance*. O recurso *churn resistance* procura melhorar o desempenho dos *hit* quando o *buffer* lida com um conjunto de páginas maior que o tamanho do *buffer* que são acessados repetidamente com a mesma frequência.

3.1.12 Substituição baseada na abordagem Seq2Seq

Hyejeong Choi e Sejin Park apresentam um algoritmo de substituição usando uma solução de aprendizado de máquina com foco na abordagem Seq2Seq baseada na rede neural LSTM (CHOI; PARK, 2022). Em resumo, o algoritmo faz previsões das requisições de páginas futuras e seleciona como candidatas as vítimas que não constam na previsão. Sua arquitetura é composta por seis componentes:

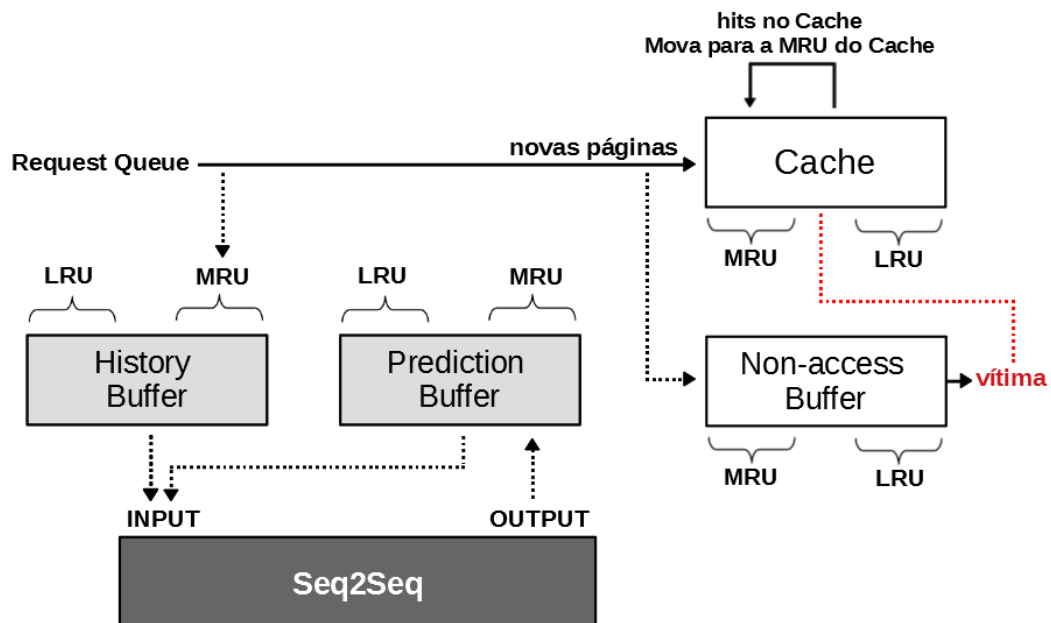
- *Cache*: Este componente é a base do algoritmo de substituição. Funciona de forma semelhante ao algoritmo LRU. Novas páginas são inseridas no lado da MRU e quando ocorre uma *hit*, a página é movida para o lado da MRU. No entanto, difere na seleção da vítima.
- *Seq2Seq*: Componente responsável por processar o modelo e fazer previsões. A abordagem recebe como entrada um conjunto de informações de requisições de páginas de tamanho fixo. Este conjunto de entrada considera o identificador de página (número do bloco), frequência, *reuse distance* e delta, onde delta representa a diferença entre solicitações consecutivas. A abordagem então gera a previsão da próxima solicitação futura com o

possível identificador de página e o delta aproximado, baseada na abordagem Seq2Seq

- *Request Queue* e *History buffer*: A *Request Queue* captura os metadados da últimas requisições por página ao *buffer* na ordem temporal e usa essas informações para preencher o *History buffer* com um tamanho igual ao conjunto de entrada do Seq2Seq.
- *Prediction buffer*: Essa estrutura tem tamanho igual ao do Cache e é responsável por armazenar as informações de saída do Seq2Seq.
- *Non-access buffer*: Mantém as páginas que serão o alvo como candidatas à vítimas dependendo do processo de predição. O algoritmo julga que essas páginas não serão solicitadas no futuro.

O *Prediction buffer* e *History buffer* mantêm apenas metadados de páginas que são gerados pelo Seq2Seq ou obtidos pela *Request Queue*. O componente Seq2Seq preenche o *Prediction buffer* inicialmente usando o *History buffer* como entrada. Porém, como a saída é limitada e são necessárias mais informações de entrada para o modelo, a estratégia opta por usar as informações de predições anteriores armazenadas no *Prediction buffer* como entrada para o Seq2Seq. Isso gera uma relação de dependência entre as predições no conjunto. Se uma única predições estiver errada, predições consecutivas também podem estar.

Figura 18 – Substituição baseada na abordagem Seq2Seq fluxo de execução



Fonte: O Autor.

A Figura 18 descreve fluxo de execução do algoritmo. Quando ocorre um *hit*, a página é movida para a **MRU** do *Cache*. O algoritmo faz predições para preencher o *Prediction*

buffer caso ocorra uma falha (*miss*) e o *buffer* esteja cheio. Em seguida, atualiza o *Non-access buffer* removendo as páginas presentes no *Prediction buffer*. Logo, de acordo com o algoritmo, o *Non-access buffer* reside as informações de páginas que não devem ser requisitadas ao *buffer* em um futuro próximo. A página LRU do *Non-access buffer* é escolhida como vítima e é removida das estruturas *Cache* e *Non-access buffer*.

A abordagem também verifica se a predição está correta ao comparar novas requisições com o *Prediction buffer*. Quando o número de erros excede um determinado limite, o *Prediction buffer* é limpo e novas predições são feitas.

3.2 Políticas de substituição de *buffer* assimétricas

Esta seção foca nas políticas de substituição de *buffer* assimétricas. Com o emergência de novas tecnologias de armazenamento, surgiu uma nova classe de algoritmos de substituição. Essas estratégias agora consideram o tipo de operação (leitura/gravação) para aproveitar a assimetria da mídia de armazenamento, melhorando significativamente o desempenho. Algumas estratégias discutidas na Seção 3.1 também foram adaptadas para trabalhar com essa nova tecnologia de armazenamento.

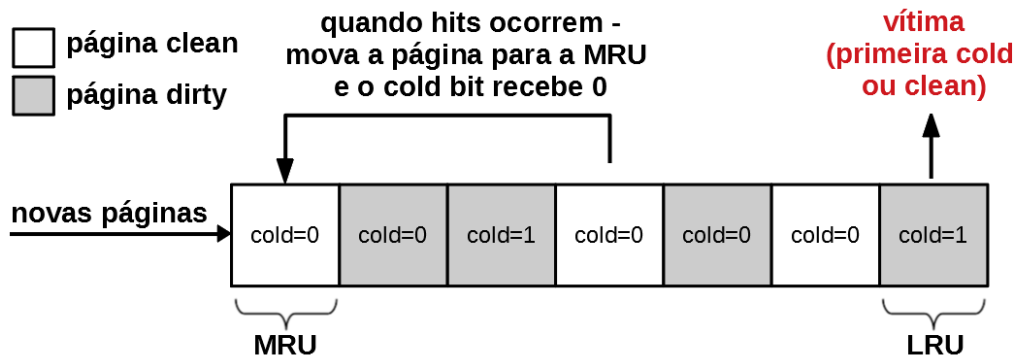
Dividir o *buffer* em áreas de leitura e gravação é uma abordagem simples que oferece uma nova perspectiva no desenvolvimento de novos algoritmos. No entanto, apenas esta simples divisão não é suficiente para lidar com as questões de *hit ratio* e diferentes padrões de acesso. Outros aspectos, como a frequência de páginas, motivam combinações adicionais de algoritmos e estruturas para equilibrar o *hit ratio* e diminuir o número de gravações.

3.2.1 CFLRU

Clean-First LRU (CFLRU) foi desenvolvido para trabalhar com mídia assimétrica. É a primeira estratégia endereçada à memória flash. O CFLRU tenta despejar primeiro as páginas leitura (*clean*), evitando assim o custo das operações de escrita (*dirty*) durante as transações (PARK *et al.*, 2006).

A CFLRU divide a LRU em duas listas, chamadas de regiões *working* e *clean-first*. O parâmetro de ajuste W representa o tamanho *clean-first region* em páginas. O valor de W pode ser estático ou dinâmico. O método dinâmico calcula o valor de W coletando periodicamente estatísticas de operações de leitura e escrita e considera o custo dessas operações sobre a mídia

Figura 20 – LRU-WSR fluxo de execução



Fonte: O Autor.

segunda chance, esta página então é movida para o lado MRU, e o *cold bit* é definido como 1. O processo é então repetido para testar uma página na posição LRU.

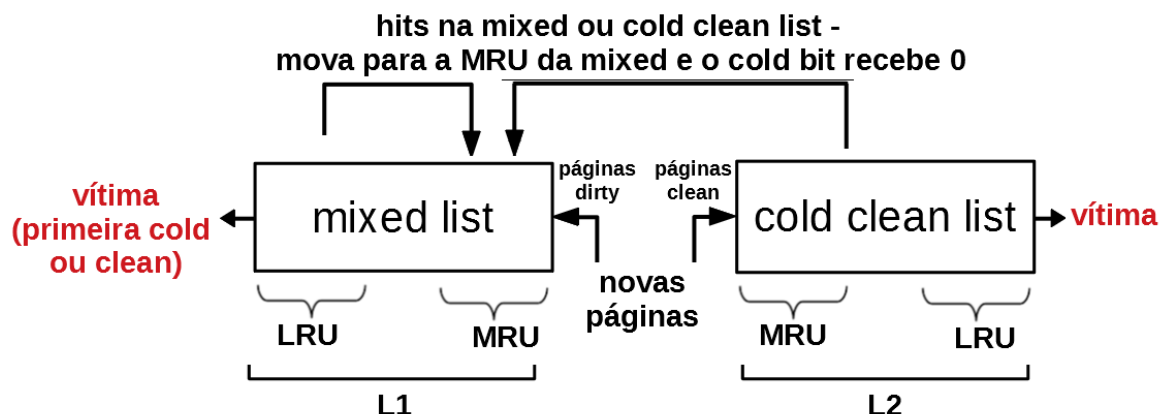
- Caso 3: Se a página LRU é *dirty* com um *cold bit* igual a 1. A página é despejada.

O LRU-WSR evita que páginas *dirty* permaneçam por muito tempo no *buffer* usando o status do *cold bit*. No entanto, sua desvantagem é que a complexidade não é constante, pois há casos em que é necessário mover várias páginas *dirty* para o lado da MRU até encontrar uma página *cold* ou *clean*.

3.2.3 CCF-LRU

Cold-Clean-First LRU (CCF-LRU) é baseado nos algoritmos LRU-WSR e CFLRU (ver Seções 3.2.2 e 3.2.1). Essa abordagem dará às páginas *clean* uma segunda chance ao escolher as vítimas. O CCF-LRU contém duas listas, *mixed* e *cold-clean*, com tamanhos dinâmicos L1 e L2, respectivamente. Cada página também tem seu *cold bit* semelhante ao LRU-WSR (LI *et al.*, 2009).

Figura 21 – CCF-LRU fluxo de execução



Fonte: O Autor.

A Figura 21 descreve o fluxo de execução do CCF-LRU. Novas páginas são inseridas com um *cold bit* igual a 1 na MRU da lista *cold-clean*. O CCF-LRU foi proposto para sistemas baseados em memória flash. Inicialmente, as novas páginas são inseridas como páginas *clean* na lista *cold-clean*, então quando as páginas recebem uma atualização, elas são consideradas *dirty* e movidas para a lista *mixed*.

Quando ocorrem *hits* em ambas as listas, a página é movida para a MRU da lista *mixed* e o *cold bit* é definido para 0. Quando o *buffer* está cheio (isto é, Capacidade do *buffer* = $L1 + L2$), o algoritmo remove a LRU da lista *cold-clean*. Todavia, se esta lista estiver vazia, o algoritmo verifica o LRU da lista *mixed*, e há três casos possíveis:

- Caso 1: Se a página LRU tem um *cold bit* igual a 1. A página é imediatamente despejada independentemente de ser *dirty* ou *clean*.
- Caso 2: Se a página LRU é *clean* com um *cold bit* igual a 0. A página recebe uma segunda chance e é movida para a MRU da lista *cold-clean* com o *cold bit* igual a 1, e o procedimento é executado novamente para uma nova LRU da lista *mixed*.
- Caso 3: Se a página LRU é *dirty* com o *cold bit* igual a 1. A página recebe uma segunda chance e é movida para a MRU da lista *mixed* com o *cold bit* igual a 1, e o procedimento é executado novamente para uma nova LRU de a lista *mixed*.

Fica evidente que o CCF-LRU não possui complexidade constante, pois em cenários específicos quando a lista *cold-clean* está vazia, é necessário encontrar uma página *cold* na lista *mixed*. Apesar disso, há outro problema de *cache starvation* (ver Seção 2.3.2). Quando a lista *cold-clean* estiver vazia, as novas páginas inseridas na *cold-clean* sempre serão indicadas como vítimas imediatamente, evitando que essa lista cresça e assim podendo diminuir o *hit ratio*.

Controllable Cold Clean First Least Recently Used (CCCF-LRU) é uma melhoria do CCF-LRU. A diferença é que faz uso do parâmetro *minCCL* que determina o tamanho mínimo da lista *cold-clean*, evitando que a lista fique muito pequena. Se este tamanho mínimo for alcançado, o algoritmo procura uma vítima na lista *mixed* (XIA; BU, 2015).

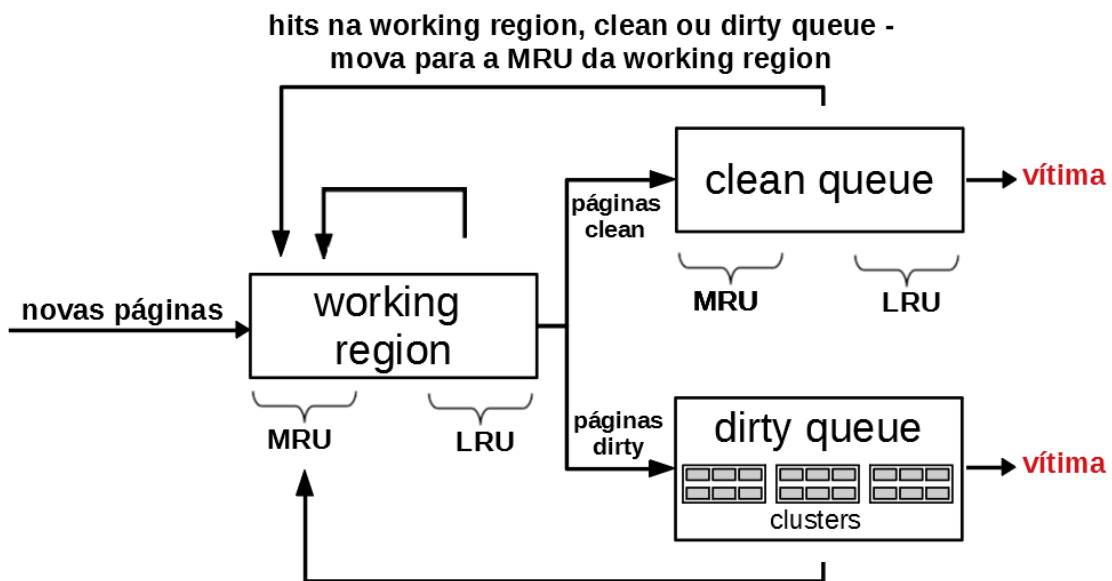
3.2.4 CFDC

Clean-First Dirty-Clustered (CFDC) é um algoritmo baseado no CFLRU (ver Seção 3.2.1) e explora melhorias no desempenho da memória flash usando técnicas baseadas em blocos semelhantes aos algoritmos *Block-Level LRU* (BPLRU) (KIM; AHN, 2008) e *Flash-Aware buffer* (FAB) (JO *et al.*, 2006). Em resumo, esses algoritmos usam estratégias para agrupar páginas,

melhorando assim o desempenho da memória flash (OU *et al.*, 2009).

O CFDC é composto pela *working region* e *priority region*. A *working region* pode ser implementada usando outro algoritmo de *buffer*, por exemplo, LRU. A *priority region* é composta pelas filas *clean* e *dirty*. O parâmetro de ajuste *priority window* determina o tamanho da região de prioridade. Durante os testes, os autores usam 50% do tamanho total do *buffer* para cada região. Os tamanhos de fila *clean* e *dirty* são dinâmicos, mas a soma das filas não excede o valor da *priority window*.

Figura 22 – CFDC fluxo de execução



Fonte: O Autor.

A Figura 22 descreve o fluxo de execução do CFDC. Novas páginas são inseridas na *working region*. Quando a *working region* está cheia, a sua página LRU é movida para filas *clean* ou *dirty* de acordo com seu tipo de operação. A fila *dirty* é uma fila de prioridade de *clusters*. O CFDC usa uma estrutura de *hash* para mapear em qual *cluster* uma página deve residir. O algoritmo também calcula uma prioridade para cada *cluster* que determina a importância do *cluster* para o algoritmo.

Quando ocorre *hits* em qualquer uma das estruturas, as páginas são movidas para a MRU da *working region*. Quando é necessário escolher uma vítima, se a fila *clean* não estiver vazia, o algoritmo seleciona a LRU como vítima. Caso contrário, o algoritmo despeja a primeira página do *cluster* com a prioridade mais baixa da fila *dirty*. Em seguida, esse *cluster* é marcado para que suas páginas sejam as próximas vítimas até que o *cluster* esteja vazio. Em vista disso, há uma alta probabilidade de que as páginas no mesmo *cluster* sejam liberadas para o mesmo

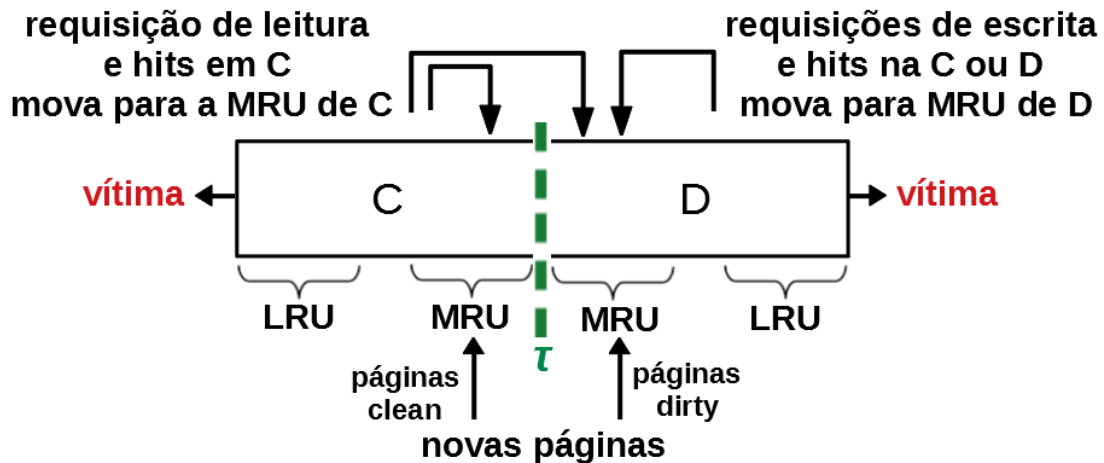
bloco de memória flash por conta da proximidade dos seus identificadores dentro do *cluster*.

A complexidade do CFDC não é constante devido aos mecanismos de agrupamento. No entanto, ele também apresenta o problema de cache starvation (ver Seção 2.3.2) semelhante ao algoritmo CCF-LRU (consulte a Seção 3.2.3). Quando a fila *clean* estiver vazia e uma nova página *clean* for inserida, a próxima vítima será imediatamente aquela página recente, evitando que a fila *clean* cresça podendo diminuir o desempenho no *hit ratio*.

3.2.5 CASA

Cost-Aware Self-Adaptive (CASA) é uma política de substituição de páginas para mídia assimétrica. O CASA divide o *buffer* em duas regiões com tamanhos dinâmicos, $|C|$ e $|D|$, que são listas *clean* e *dirty*, respectivamente. Em que $|C| + |D| = B$, sendo B o tamanho do *buffer* em páginas. O algoritmo usa o *advisor* τ para controlar o tamanho das listas. O CASA usa dois parâmetros de ajuste, C_R e C_W , que representam o custo das operações de leitura ou escrita e são usados quando o *advisor* τ é ajustado (OU; HÄRDER, 2010).

Figura 23 – CASA fluxo de execução



Fonte: O Autor.

A Figura 23 descreve o fluxo de execução do CASA. Novas páginas são inseridas na MRU de C ou D, dependendo do tipo de operação. Para selecionar uma vítima, o CASA verifica o *advisor* τ . Se $|C| > \tau$, então a vítima será a LRU de C. Caso contrário, a LRU de D. Quando ocorre um *hit*, o algoritmo tem quatro casos:

- Caso 1: *Hits* em C com a operação de leitura. A página é movida para o MRU de C e o *advisor* τ é ajustado para $\tau = \min(\tau + C_R \times (|D| / |C|), B)$.
- Caso 2: *Hits* em D com a operação de escrita. A página é movida para o MRU de D e o

página é movida para a MRU da fila *hot* e o *cold bit* é definido como 0. Se o *hit* foi na fila *hot*, somente o FC da fila *hot* é ajustado. Caso contrário, o AD-LRU ajusta o FC das duas filas.

Para substituir uma página o AD-LRU primeiro seleciona uma das duas filas usando o parâmetro *min_lc* quando o *buffer* está cheio. A fila *cold* é selecionada como vítima se $C > min_lc$, em que C é o número de páginas da fila *cold*. Caso contrário, o algoritmo seleciona a fila *hot*. Em seguida, se o ponteiro FC da fila escolhida não for nulo, o algoritmo seleciona a página referenciada pelo ponteiro FC como vítima e depois ajusta o ponteiro FC para referenciar uma nova página *clean* desta fila caso exista.

Se FC for nulo, o algoritmo deve remover a página LRU *dirty*. No entanto, se a página tem um *cold bit* igual a 1, semelhante ao LRU-WSR, o algoritmo dá uma segunda chance, e define o *cold bit* como 0 e move a página para o lado MRU da fila selecionada, então o processo é repetido. Se o *cold bit* for igual a 0, a página LRU *dirty* será finalmente despejada.

O parâmetro *min_lc* controla o tamanho da fila *cold*, evitando que ela fique muito pequena ou fique muito grande. Consequentemente, *min_lc* evita o *cache starvation* e fornece uma *scan resistance* (ver Seções 2.3.2 e 2.3.1). O AD-LRU não possui complexidade constante devido à heurística da segunda chance, semelhante ao LRU-WSR.

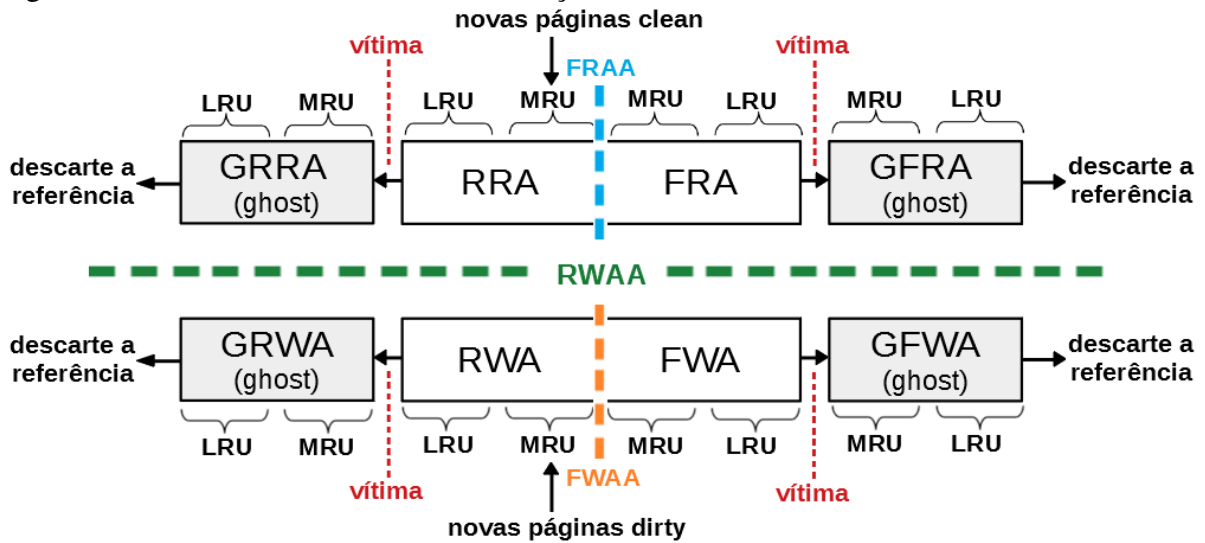
3.2.7 SCMBP-SCCW

SCMBP-SCCW é uma política de substituição para mídia assimétrica. O algoritmo é baseado em *Hierarchical Adaptive Replacement Cache* (H-ARC) (FAN *et al.*, 2014) e no ARC (ver Seção 3.1.9) com características de CFDC (ver Seção 3.2.4). O SCMBP-SCCW utiliza os benefícios dos mecanismos de adaptação do ARC (recência e frequência) aplicados para se adaptar às mudanças na carga de trabalho como a leitura e escrita de páginas. Ele também usa o mecanismo de agrupamento CFDC na sua *Recency Write Area* (RWA) (TAVARES, 2015).

A ideia é usar dois ARCs chamados RR (*Read Region*) e WR (*Write Region*). O *buffer* é dividido em quatro regiões com quatro listas *ghosts* auxiliares. O *advisor* externo RWAA (*Read/Write Area Advisor*) controla os tamanhos de RR e WR. Os consultores internos FRAA (*Frequency Read Area Advisor*) e FWAA (*Frequency Write Area Advisor*) controlam os tamanhos das regiões de recência e frequência.

A Figura 25 descreve o fluxo de execução do SCMBP-SCCW. Novas páginas são inseridas na MRU da RRA (*Recent Read Area*) ou na RWA (*Recency Write Area*) dependendo do tipo de operação (leitura/gravação) e o *advisor* RWAA é ajustado. Quando ocorre um *hit*, a

Figura 25 – SCMBP-SCCW fluxo de execução



Fonte: O Autor.

página é movida para FRA (*Frequency Read Area*) ou FWA (*Frequency Write Area*), dependendo do tipo de operação. Se o *hit* foi nas listas *ghosts*, a página é carregada e o *advisor* RWAA é ajustado, junto com os *advisors* FRAA ou FWAA, dependendo da operação.

Na seleção da vítima, o algoritmo primeiro escolhe entre RR ou WR usando o *advisor* RWAA. Em seguida, os *advisors* internos FRAA ou FWAA decidem entre recência ou frequência despejando a página LRU. No entanto, se a vítima residir no RWA, as páginas do *cluster* com menor prioridade são selecionadas como vítimas durante as solicitações subsequentes, semelhante ao CFDC.

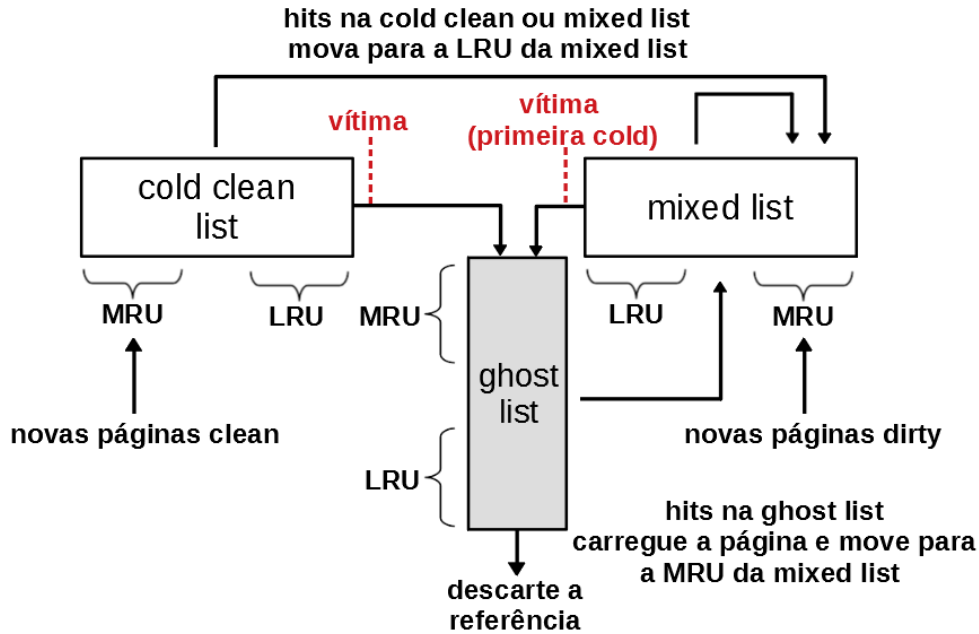
Ao atualizar os *advisors*, o algoritmo usa vários parâmetros de ajuste que ajudam a regular os tamanhos das regiões. Portanto, é possível impulsionar as regiões de escrita a crescerem mais rapidamente do que as de leitura, reduzindo o número de escritas e favorecendo mídias assimétricas. Da mesma forma que o ARC, o SCMBP-SCCW possui mecanismos de *scan resistance* (ver Seção 2.3.1), mas não possui uma complexidade constante devido aos mecanismos de agrupamento.

3.2.8 GASA

Ghost buffer Assisted and Self-tuning Algorithm (GASA) é uma estratégia na qual o *buffer* é dividido em duas listas, *cold clean* e *mixed*, e usa uma lista *ghost* auxiliar. Os tamanhos das três listas são ajustados dinamicamente e cada página tem uma *hot-flag* e uma *ghost-flag*. A *hot-flag* é usada para dar uma segunda chance às páginas *hot*, segundo o algoritmo. A *ghost-flag*

é usado para ajustar o tamanho da lista *ghost* GS, que usa os parâmetros de ajuste *GSMIN* e *GSMAX*, indicando o tamanho mínimo e máximo (LI *et al.*, 2016).

Figura 26 – GASA fluxo de execução



Fonte: O Autor.

A Figura 26 descreve o fluxo de execução do GASA. Novas páginas são inseridas na MRU da lista *cold clean* ou *mixed* dependendo do tipo de operação com *hot-flag* e *ghost-flag* igual a 0. Para *hits* na lista *cold clean* ou *mixed*, a página é movida para a MRU da lista *mixed* e a *hot-flag* é definida como 1. Porém, se a página tiver uma *ghost-flag* igual a 1, o tamanho da lista *ghost* é incrementado $GS = \min(GS + 1, GSMAX)$ e a *ghost-flag* é definida como 0. Para *hits* na lista *ghost*, a página é carregada na memória e movida para o MRU da lista *mixed* e a *ghost-flag* e *hot-flag* são definidas como 1.

Quando o *buffer* está cheio e uma vítima precisa ser escolhida, se a lista *cold clean* não estiver vazia, o algoritmo seleciona a sua LRU como vítima. Caso contrário, o algoritmo seleciona o LRU da lista *mixed* como vítima se o *hot-flag* for igual a 0. Se o *hot-flag* for igual a 1, o GASA lhe dará uma segunda chance, atualizando a *hot-flag* para 0 e movendo a página para uma lista *cold clean* ou *mixed*, dependendo do tipo de operação. O processo é repetido para a nova LRU da lista *cold clean* ou *mixed*. Após selecionar uma vítima, seus metadados são inseridos no MRU da lista *ghost*. No entanto, se a vítima tiver a *ghost-flag* igual a 1, o tamanho da lista *ghost* é decrementado $GS = \max(GSMIN, GS - GSMAX/(GSMAX-GS+1))$.

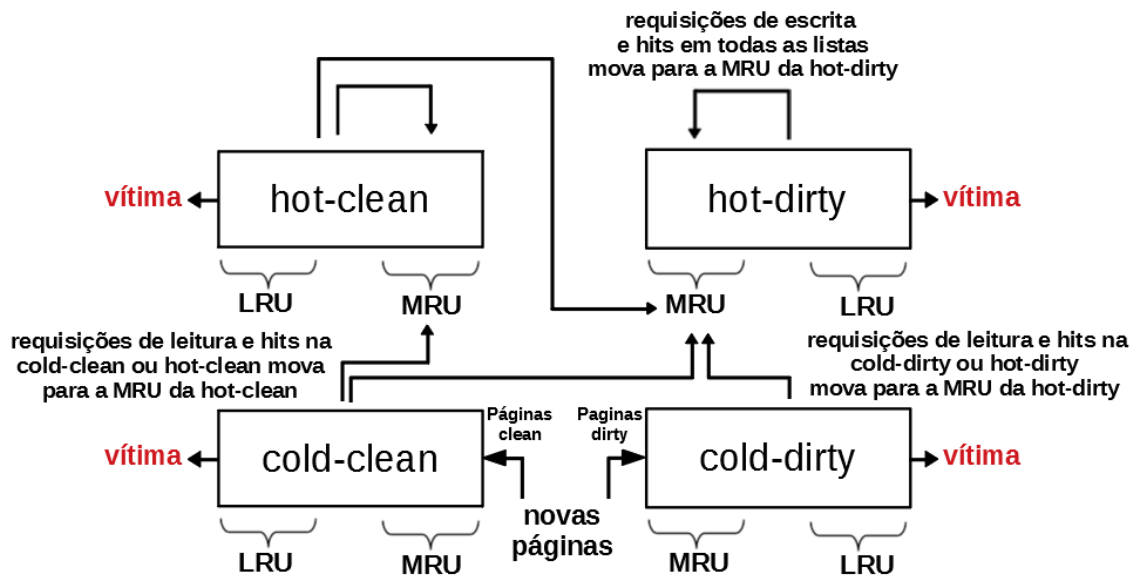
O GASA não tem uma complexidade constante devido ao processo de escolha da

vítima. O algoritmo propõe ajustar o tamanho da lista de *ghost*. Se o tamanho for pequeno, pode dificultar a identificação das páginas *hot*. Se o tamanho for grande, as páginas da lista *cold clean* terão uma alta probabilidade de serem identificadas como *hot* segundo o algoritmo, o que pode aumentar o número de gravações.

3.2.9 LLRU

Locality-aware Least Recently Used (LLRU) é uma política de substituição que divide o *buffer* em quatro listas LRU: *cold-clean*, *cold-dirty*, *hot-clean* e *hot-dirty*. O tamanho de cada lista é dinâmico e cada página possui um contador de referência. O algoritmo usa dois parâmetros de ajuste: E_{CC} e E_{CD} , que correspondem ao custo de leitura e gravação respectivamente (HE *et al.*, 2017).

Figura 27 – LLRU fluxo de execução



Fonte: O Autor.

A Figura 27 descreve o fluxo de execução da LLRU. Novas páginas são inseridas na *cold-clean* ou *cold-dirty*, dependendo do tipo de operação. Quando ocorre um *hit*, a página é movida para a MRU da *hot-dirty* se a operação for de escrita em qualquer uma das quatro listas. Se a operação for leitura e ocorre um *hit* na lista *cold-clean* ou *hot-clean*, a página será movida para a MRU da *hot-clean*. Por fim, se a operação for leitura e ocorrer um *hit* na *cold-dirty* ou *hot-dirty*, a página será movida para o MRU da *hot-dirty*.

O algoritmo primeiro calcula o custo da página (PC) para cada LRU das quatro listas para selecionar uma vítima. PC é calculado pela fórmula 3.2, em que AT representa o número de

acessos dessa página. Entre as quatro páginas LRU em cada lista, a vítima será a página com o menor PC .

$$PC(p) = \begin{cases} AT(p) * E_{CC}, & \text{if } p \text{ is } clean. \\ AT(p) * E_{CD}, & \text{if } p \text{ is } dirty. \end{cases} \quad (3.2)$$

O LLRU tem complexidade constante e uma das suas desvantagens é que pode sofrer com o problema de *cache starvation* (ver Seção 2.3.2). As páginas da lista *cold-clean* têm alta probabilidade de sempre serem selecionadas como vítimas, impedindo que a lista cresça. O AM-LRU (WU *et al.*, 2019) foi proposto para melhorar o algoritmo LLRU usando um parâmetro de ajuste chamado *min_len*, semelhante ao AD-LRU (ver Seção 3.2.6). O *min_len* controla o tamanho da *cold-clean*, evitando que a lista fique muito pequena.

REFERÊNCIAS

- ARI, I.; AMER, A.; GRAMACY, R. B.; MILLER, E. L.; BRANDT, S. A.; LONG, D. D. Acme: Adaptive caching using multiple experts. In: **WDAS**. [S.l.: s.n.], 2002. v. 2, p. 143–158.
- ARLITT, M.; CHERKASOVA, L.; DILLEY, J.; FRIEDRICH, R.; JIN, T. Evaluating content management techniques for web proxy caches. **SIGMETRICS Perform. Eval. Rev.**, Association for Computing Machinery, New York, NY, USA, v. 27, n. 4, p. 3–11, mar. 2000. ISSN 0163-5999. Disponível em: <<https://doi.org/10.1145/346000.346003>>.
- ASSOCIATION, S. N. I. **Microsoft Storage File Server Traces**. 2007. <<http://iotta.snia.org/traces/158>>. [Online; accessed 04-October-2014].
- ASSOCIATION, S. N. I. **TPC-C SNIA Traces**. 2007. <<http://iotta.snia.org/traces/131>>. [Online; accessed 10-October-2014].
- ASSOCIATION, S. N. I. **TPC-E SNIA Traces**. 2007. <<http://iotta.snia.org/traces/133>>. [Online; accessed 10-October-2014].
- BAI, S.; BAI, X.; CHE, X. Window-lrfu: A cache replacement policy subsumes the lru and window-lfu policies. John Wiley and Sons Ltd., GBR, v. 28, n. 9, p. 2670–2684, jun. 2016. ISSN 1532-0626. Disponível em: <<https://doi.org/10.1002/cpe.3730>>.
- BANSAL, S.; MODHA, D. S. Car: Clock with adaptive replacement. In: **Proceedings of the 3rd USENIX Conference on File and Storage Technologies**. USA: USENIX Association, 2004. (FAST '04), p. 187–200.
- BARROS, A.; LEHFELD, N. **Fundamentos de Metodologia Científica - um Guia Para a Iniciação Científica**. [S.l.]: Makron Books, 2000.
- Belady, L. A. A study of replacement algorithms for a virtual-storage computer. **IBM Systems Journal**, v. 5, n. 2, p. 78–101, 1966. ISSN 0018-8670.
- CHOI, H.; PARK, S. Learning future reference patterns for efficient cache replacement decisions. **IEEE Access**, v. 10, p. 25922–25934, 2022.
- CORBATÓ, F.; TECHNOLOGY), P. M. M. I. of. **A Paging Experiment with the Multics System**. [S.l.]: Massachusetts Institute of Technology, 1968. (Project MAC).
- DIACONU, C.; FREEDMAN, C.; ISMERT, E.; LARSON, P.; MITTAL, P.; STONECIPHER, R.; VERMA, N.; ZWILLING, M. Hekaton: Sql server's memory-optimized oltp engine. In: **ACM International Conference on Management of Data 2013**. [S.l.: s.n.], 2013.
- EFFELSBERG, W.; HÄRDER, T. Principles of database buffer management. **ACM Trans. Database Syst.**, ACM, New York, NY, USA, 1984.
- ELDAWY, A.; LEVANDOSKI, J.; LARSON, P.-r. Trekking through siberia: Managing cold data in a memory-optimized database. **Proc. VLDB Endow.**, VLDB Endowment, v. 7, n. 11, p. 931–942, jul 2014. ISSN 2150-8097. Disponível em: <<https://doi.org/10.14778/2732967.2732968>>.
- FAN, Z.; DU, D. H. C.; VOIGT, D. H-arc: A non-volatile memory based cache policy for solid state drives. In: **2014 30th Symposium on Mass Storage Systems and Technologies (MSST)**. [S.l.: s.n.], 2014. p. 1–11.

GARCIA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. **Implementação de sistemas de banco de dados**. [S.l.]: Campus, 2001.

HARIZOPOULOS, S.; ABADI, D. J.; MADDEN, S.; STONEBRAKER, M. Oltp through the looking glass, and what we found there. In: **Proceedings of the 2008 ACM SIGMOD**. [S.l.: s.n.], 2008. (SIGMOD '08).

HE, J.; JIA, G.; HAN, G.; WANG, H.; YANG, X. Locality-aware replacement algorithm in flash memory to optimize cloud computing for smart factory of industry 4.0. **IEEE Access**, v. 5, p. 16252–16262, 2017.

HELLERSTEIN, J. M.; STONEBRAKER, M.; HAMILTON, J. Architecture of a database system. **Found. Trends databases**, Now Publishers Inc., Hanover, MA, USA, v. 1, n. 2, p. 141–259, fev. 2007. ISSN 1931-7883.

JIANG, S.; CHEN, F.; ZHANG, X. Clock-pro: An effective improvement of the clock replacement. In: **Proceedings of the Annual Conference on USENIX Annual Technical Conference**. USA: USENIX Association, 2005. (ATEC '05), p. 35.

JIANG, S.; ZHANG, X. Lirs: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In: **Proceedings of the 2002 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems**. New York, NY, USA: Association for Computing Machinery, 2002. (SIGMETRICS '02), p. 31–42. ISBN 1581135319. Disponível em: <<https://doi.org/10.1145/511334.511340>>.

JIN, P.; OU, Y.; HÄRDER, T.; LI, Z. Ad-lru: An efficient buffer replacement algorithm for flash-based databases. **Data Knowl. Eng.**, Elsevier Science Publishers B. V., NLD, v. 72, p. 83–102, fev. 2012. ISSN 0169-023X. Disponível em: <<https://doi.org/10.1016/j.datak.2011.09.007>>.

JO, H.; KANG, J.-U.; PARK, S.-Y.; KIM, J.-S.; LEE, J. Fab: flash-aware buffer management policy for portable media players. **IEEE Transactions on Consumer Electronics**, v. 52, n. 2, p. 485–493, 2006.

JR, E. S. G. Exponential smoothing: The state of the art—part ii. **International journal of forecasting**, Elsevier, v. 22, n. 4, p. 637–666, 2006.

JUNG, H.; SHIM, H.; PARK, S.; KANG, S.; CHA, J. Lru-wsr: integration of lru and writes sequence reordering for flash memory. **IEEE Transactions on Consumer Electronics**, v. 54, n. 3, p. 1215–1223, 2008.

JUNG, H.; YOON, K.; SHIM, H.; PARK, S.; KANG, S.; CHA, J. Lirs-wsr: Integration of lirs and writes sequence reordering for flash memory. In: GERVASI, O.; GAVRILOVA, M. L. (Ed.). **Computational Science and Its Applications – ICCSA 2007**. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007. p. 224–237. ISBN 978-3-540-74472-6.

KABRA, N.; DEWITT, D. J. Efficient mid-query re-optimization of sub-optimal query execution plans. In: ACM. **ACM SIGMOD Record**. [S.l.], 1998.

KIM, C.-S.; LEE, D.; KIM, J.-H.; CHOI, J.; NOH, S.; MIN, S.; CHO, Y. Lrfu (least recently/frequently used) replacement policy: A spectrum of block replacement policies. In: . [S.l.: s.n.], 1996.

KIM, H.; AHN, S. Bplru: A buffer management scheme for improving random writes in flash storage. In: **Proceedings of the 6th USENIX Conference on File and Storage Technologies**. USA: USENIX Association, 2008. (FAST'08).

LEVANDOSKI, J. J.; LARSON, P.-Å.; STOICA, R. Identifying hot and cold data in main-memory databases. In: IEEE. **2013 IEEE 29th International Conference on Data Engineering (ICDE)**. [S.l.], 2013. p. 26–37.

LI, C. Dlirs: Improving low inter-reference recency set cache replacement policy with dynamics. In: **Proceedings of the 11th ACM International Systems and Storage Conference**. New York, NY, USA: Association for Computing Machinery, 2018. (SYSTOR '18), p. 59–64. ISBN 9781450358491. Disponível em: <<https://doi.org/10.1145/3211890.3211891>>.

LI, C. Clock-pro+: Improving clock-pro cache replacement with utility-driven adaptation. In: **Proceedings of the 12th ACM International Conference on Systems and Storage**. New York, NY, USA: Association for Computing Machinery, 2019. (SYSTOR '19), p. 1–7. ISBN 9781450367493. Disponível em: <<https://doi.org/10.1145/3319647.3325838>>.

LI, C.; FENG, D.; HUA, Y.; XIA, W.; WANG, F. Gasa: A new page replacement algorithm for nand flash memory. In: **2016 IEEE International Conference on Networking, Architecture and Storage (NAS)**. [S.l.: s.n.], 2016. p. 1–9.

LI, Z.; JIN, P.; SU, X.; CUI, K.; YUE, L. Ccf-lru: a new buffer replacement algorithm for flash memory. **IEEE Transactions on Consumer Electronics**, v. 55, n. 3, p. 1351–1359, 2009.

LIN, M.; CHEN, S.; WANG, G.; WU, T. Hdc: An adaptive buffer replacement algorithm for nand flash memory-based databases. **Optik**, v. 125, n. 3, p. 1167–1173, 2014. ISSN 0030-4026. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0030402613011959>>.

MEGIDDO, N.; MODHA, D. S. Arc: A self-tuning, low overhead replacement cache. In: **Proceedings of the 2Nd USENIX Conference on File and Storage Technologies**. [S.l.: s.n.], 2003.

NICOLA, V. F.; DAN, A.; DIAS, D. M. Analysis of the generalized clock buffer replacement scheme for database transaction processing. In: . New York, NY, USA: Association for Computing Machinery, 1992. (SIGMETRICS '92/PERFORMANCE '92), p. 35–46. ISBN 0897915070. Disponível em: <<https://doi.org/10.1145/133057.133084>>.

ON, S. T.; LI, Y.; HE, B.; WU, M.; LUO, Q.; XU, J. Fd-buffer: A buffer manager for databases on flash disks. In: . New York, NY, USA: Association for Computing Machinery, 2010. (CIKM '10), p. 1297–1300. ISBN 9781450300995. Disponível em: <<https://doi.org/10.1145/1871437.1871605>>.

O'NEIL, E. J.; O'NEIL, P. E.; WEIKUM, G. The lru-k page replacement algorithm for database disk buffering. **SIGMOD Rec.**, ACM, New York, NY, USA, v. 22, n. 2, p. 297–306, jun. 1993. ISSN 0163-5808. Disponível em: <<http://doi.acm.org/10.1145/170036.170081>>.

ORACLE. **MySQL 8.0 Reference Manual**. 2021. Disponível em: <<https://dev.mysql.com/doc/refman/8.0/en/innodb-buffer-pool.html>>.

ORACLE. **Oracle - Database Concepts**. 2021. Disponível em: <<https://docs.oracle.com/en/database/oracle/oracle-database/21/cncpt/memory-architecture.html#GUID-AA95A3C7-C452-4DDB-B7E4-44CB43B1E9C3>>.

OU, Y.; HÄRDER, T. Clean first or dirty first? a cost-aware self-adaptive buffer replacement policy. In: . New York, NY, USA: Association for Computing Machinery, 2010. (IDEAS '10), p. 7–14. ISBN 9781605589008. Disponível em: <<https://doi.org/10.1145/1866480.1866482>>.

OU, Y.; HÄRDER, T.; JIN, P. Cfdc: A flash-aware replacement policy for database buffer management. In: **Proceedings of the Fifth International Workshop on Data Management on New Hardware**. New York, NY, USA: Association for Computing Machinery, 2009. (DaMoN '09), p. 15–20. ISBN 9781605587011. Disponível em: <<https://doi.org/10.1145/1565694.1565698>>.

OU, Y.; JIN, P.; HÄRDER, T. Flash-aware buffer management for database systems. **Int. J. Knowledge-Based Organ.**, IGI Global, USA, v. 3, n. 4, p. 22–39, out. 2013. ISSN 2155-6393. Disponível em: <<https://doi.org/10.4018/ijkbo.2013100102>>.

PARK, S.; KIM, Y.; URGAKONKAR, B.; LEE, J.; SEO, E. A comprehensive study of energy efficiency and performance of flash-based ssd. In: . [S.l.: s.n.], 2011. v. 57, n. 4, p. 354–365. ISSN 1383-7621.

PARK, S.-y.; JUNG, D.; KANG, J.-u.; KIM, J.-s.; LEE, J. Cflru: A replacement algorithm for flash memory. In: **Proceedings of the 2006 International Conference on Compilers, Architecture and Synthesis for Embedded Systems**. New York, NY, USA: Association for Computing Machinery, 2006. p. 234–241. ISBN 1595935436. Disponível em: <<https://doi.org/10.1145/1176760.1176789>>.

ROBINSON, J. T.; DEVARAKONDA, M. V. Data cache management using frequency-based replacement. In: **Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems**. New York, NY, USA: Association for Computing Machinery, 1990. (SIGMETRICS '90), p. 134–142. ISBN 0897913590. Disponível em: <<https://doi.org/10.1145/98457.98523>>.

RODRIGUEZ, L. V.; YUSUF, F.; LYONS, S.; PAZ, E.; RANGASWAMI, R.; LIU, J.; ZHAO, M.; NARASIMHAN, G. Learning cache replacement with CACHEUS. In: **19th USENIX Conference on File and Storage Technologies (FAST 21)**. USENIX Association, 2021. p. 341–354. ISBN 978-1-939133-20-5. Disponível em: <<https://www.usenix.org/conference/fast21/presentation/rodriguez>>.

SHASHA, D.; JOHNSON, T. 2q: A low overhead high performance buffer management replacement algorithm. In: **Proceedings of the Twentieth International Conference on Very Large Databases, Santiago, Chile**. [S.l.: s.n.], 1994. p. 439–450.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Database System Concepts, 5th Edition**. [S.l.]: McGraw-Hill Book Company, 2005.

SORJAMAA, A.; HAO, J.; REYHANI, N.; JI, Y.; LENDASSE, A. Methodology for long-term prediction of time series. **Neurocomputing**, Elsevier, v. 70, n. 16-18, p. 2861–2869, 2007.

STOICA, R.; AILAMAKI, A. Enabling efficient os paging for main-memory oltp databases. In: **Proceedings of the Ninth International Workshop on Data Management on New Hardware**. New York, NY, USA: Association for Computing Machinery, 2013. (DaMoN '13). ISBN 9781450321969. Disponível em: <<https://doi.org/10.1145/2485278.2485285>>.

TAVARES, J. A. **Database Buffer Management Strategies for Asymmetric Media**. Tese (Master dissertation) — Universidade de Fortaleza - Unifor, 2015.

TPC. **Benchmark™ C**. 2010. <<http://www.tpc.org/tpcc/>>. Acessado em: 22/08/2022.

VERÍSSIMO, A. J.; ALVES, C. da C.; HENNING, E.; AMARAL, C. E. do; CRUZ, A. C. da. Métodos estatísticos de suavização exponencial holt-winters para previsão de demanda em uma empresa do setor metal mecânico. **Revista Gestão Industrial**, v. 8, n. 4, 2013.

VIETRI, G.; RODRIGUEZ, L. V.; MARTINEZ, W. A.; LYONS, S.; LIU, J.; RANGASWAMI, R.; ZHAO, M.; NARASIMHAN, G. Driving cache replacement with ml-based lecar. In: **Proceedings of the 10th USENIX Conference on Hot Topics in Storage and File Systems**. USA: USENIX Association, 2018. (HotStorage'18), p. 3.

VOLTACTIVEDATA. **VOLT ACTIVE DATA**. 2010. <<https://www.voltactivedata.com/>>. Acessado em: 12/09/2022.

WANG, F.; JIANG, X.; HUANG, J.; CHEN, F. Pa-lirs: An adaptive page replacement algorithm for nand flash memory. **Electronics**, v. 9, n. 12, 2020. ISSN 2079-9292. Disponível em: <<https://www.mdpi.com/2079-9292/9/12/2172>>.

WU, X.; CAI, D.; GUAN, S. A multiple LRU list buffer management algorithm. **IOP Conference Series: Materials Science and Engineering**, IOP Publishing, v. 569, p. 052002, aug 2019. Disponível em: <<https://doi.org/10.1088/1757-899x/569/5/052002>>.

XIA, Z.; BU, T. The implementation of flash-aware buffer replacement algorithms in postgresql. In: **2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)**. [S.l.: s.n.], 2015. p. 1215–1219.

ZHANG, W. E.; SHENG, Q. Z.; TAYLOR, K.; QIN, Y. Identifying and caching hot triples for efficient rdf query processing. In: RENZ, M.; SHAHABI, C.; ZHOU, X.; CHEEMA, M. A. (Ed.). **Database Systems for Advanced Applications**. Cham: Springer International Publishing, 2015. p. 259–274. ISBN 978-3-319-18123-3.

ZHOU, Y.; PHILBIN, J.; LI, K. The multi-queue replacement algorithm for second level buffer caches. In: **Proceedings of the General Track: 2001 USENIX Annual Technical Conference**. USA: USENIX Association, 2001. p. 91–104. ISBN 188044609X.